Engineering Structures 301 (2024) 117235

Contents lists available at ScienceDirect

Engineering Structures

journal homepage: www.elsevier.com/locate/engstruct

# An improved plate deep energy method for the bending, buckling and free vibration problems of irregular Kirchhoff plates

Zhongmin Huang [a], Linxin Peng [a,b,c,*]

[a] College of Civil Engineering and Architecture, Guangxi University, Nanning, PR China
[b] Key Laboratory of Disaster Prevention and Structural Safety of Ministry of Education, Guangxi University, Nanning, PR China
[c] Guangxi Key Laboratory of Disaster Prevention and Structural Safety, Guangxi University, Nanning, PR China

ARTICLE INFO

*Keywords:*
Deep learning
Partial differential equations
Kirchhoff plate
Coordinate transformation
Finite difference approximation

ABSTRACT

An improved plate deep energy method without penalty terms is introduced to study the bending, free vibration and buckling problems of irregular Kirchhoff plates. The finite difference produced by convolution operation is employed for a faster calculation of derivatives and enforcement of boundary conditions. Most deep learning based neural network methods adopt the penalty method to impose boundary constraints, which may cause convergence issues especially when dealing with complex solution domain or mixed boundary conditions. In this paper, the R-function is utilized to build the distance function multiplied by the neural network output to meet the Dirichlet boundary constraints, while the boundary constraints expressed by derivative functions are applied by introducing virtual nodes. With such treatments, the boundary losses of a plate system can be avoided. To validate the proposed method, numerical examples considering plates with irregular shapes and mixed boundary conditions are studied, and the results are compared to those provided by the theoretical solutions, finite element method (FEM), and literature.

## 1. Introduction

Thin plate and shell structures are widely used in engineering due to their efficiency of load-carrying behavior, high strength, high stiffness, etc. [1]. And various types of plates, including circular, annular, sectoral, trapezoidal, rectangular, triangular, and skewed configurations, have found diverse applications across several engineering sectors, such as aerospace, aeronautics, automotive, mechanical, and shipbuilding industries. These structures are common in engineering and remain a current research hotspot. For example, Wu and Hung[2] utilized a mixed isoparametric finite element method (IFEM) to investigate the free vibration behavior of functionally graded (FG) graphene platelets-reinforced composite (GPLRC) toroidal shells. In a similar vein, Civalek and Avcar [3] delved into the analysis of free vibration and buckling phenomena in non-rectangular plates reinforced with irregularly distributed functionally graded carbon nanotubes. Furthermore, there have been numerous recent studies on the plate problems, as exemplified by references [4–9]. The aforementioned works also encompass investigations into irregularly shaped plates. In summary, solving problems related to plates and shells entails the use of partial differential equations (PDEs) solving techniques. Numerous numerical methods, such as the finite element method (FEM) [10], generalized finite difference method (GFDM) [11], wavelet collocation method (WCM) [12], boundary element method (BEM) [13], and differential quadrature method (DQM) [14], and so on,are available for solving these PDEs. .

In addition to the aforementioned methods, artificial neural networks (ANN), which have high fitting capabilities, were also employed to solve PDEs in the 1990 s. However, due to the hardware and software restrictions of computers, most ANN studies focused on simple PDEs such as first-order or second-order linear PDEs [15–17] at that time. Recently, various neural network base methods have been proposed to solve the strong or weak form of PDEs with the great development of deep learning. For example, the deep Galerkin method (DGM)[18] was proposed by Sirignano and Spiliopoulos to solve higher order differential equations. Physics Informed Neural Networks (PINNs) were introduced by Raissi et al.[19] for nonlinear PDEs. PINNs were typical strong form method for PDEs and many works were developed based on them [20–23]. Nevertheless, finding strong form solutions still remains challenging, and weak form solutions are more favorable in mechanical analysis. Weinan and Yu [24] developed the Deep Ritz Method (DRM) which solved PDEs in a variational form. Samaniego et al. [25]

* Corresponding author at: College of Civil Engineering and Architecture, Guangxi University, Nanning, PR China.
*E-mail address:* penglx@gxu.edu.cn (L. Peng).

https://doi.org/10.1016/j.engstruct.2023.117235
Received 31 July 2023; Received in revised form 16 October 2023; Accepted 23 November 2023
Available online 14 December 2023
0141-0296/© 2023 Elsevier Ltd. All rights reserved.

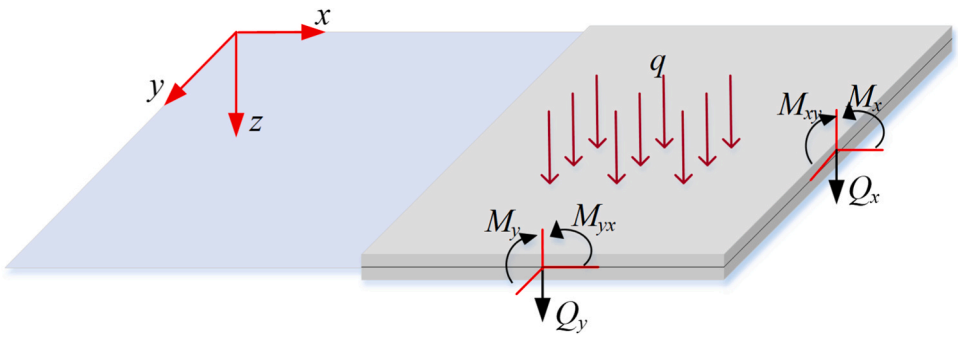



**Fig. 1.** Kirchhoff thin plate model (Cartesian coordinate system).

introduced the concept of Deep Energy Method (DEM), of which the loss function was based on the energy minimization principles. Nguyen-Thanh et al.[26] proposed a Parametric DEM (P-DEM) which has advantages, including automated normalization and the ability to represent various geometries in the physical domain while training in the parametric domain.

The aforementioned methods were applied to hyperelasticity problems [27–29], elastoplasticity problems [30,35,36], heat transfer analysis[32], topology optimization [31], etc. Recently, in the context of solving plate and shell problems using deep learning-based approaches, Bastek and Kochmann [33] solved the bending problem of shell structures using PINNs. Samaniego et al. [25] solved the bending problem of Kirchhoff plates with DEM. Sukumar and Srivastava [40] solved the governing equation of thin plate bending problem with a proper designed distance function. Zhuang et al. [34] presented the deep autoencoder based energy method for thin plate problems.

However, neural network training is a time-consuming process, and enhancing training efficiency represents another key area of research focus. For example, by dividing the solution domain into several sub domains and allocating CPU or GPU computing resources to each of them, along with parallel algorithms, the training speed was increased [37]. Based on the finite difference theory, the physics-informed geometry-adaptive convolutional neural network (PhyGeoNet) was proposed to find solution of PDEs on irregular domains [38]. The coupled automatic numerical physics informed neural network (CAN-PINN) [39] used numerical differentiation technique to speed up the training.

Overall, the general idea of the aforementioned methods is to use neural networks to approximate the solution of PDEs under specific boundary conditions (BCs), which is an optimization problem. Since boundary losses are typically difficult to avoid, the penalty method is frequently used [25,33,40]. The selection of penalty coefficients is essential to avoid convergence issues, which is still empirical. The penalty method, on the other hand, may fail for complex geometries or high order derivative boundary conditions, or it may result in a lengthy training cycle. Eliminating boundary losses can reduce the difficulty of training neural networks. To achieve this, the current dominant strategy is to construct a proper trial function (distance function) $\phi(x,\ y)$, and multiply it with the neural network approximation [40]. Nevertheless, it is still challenging to find a suitable trial function for complex boundary conditions (mixed BCs, for example).

In this paper, based on energy criterion, a penalty free deep energy method named improved plate deep energy method for Kirchhoff plate analysis is introduced. When calculating the derivatives of network output with respect to the input, the finite difference approximations performed by convolution operations are employed to improve the training efficiency. For problems with complex geometric shapes, the physical domain (the true domain of structure) is decomposed into several subdomains. Coordinate transformation is introduced to map the derivative relations between each subdomain and a square domain (the reference domain), allowing the derivatives in the physical domain to be computed using finite difference in the reference domain. In the reference domain, virtual nodes are generated for both integral operation and boundary condition enforcements. Like FEM, the restraint of displacement boundary conditions is adequate for the weak form solution of plate analysis. In order to apply the Dirichlet BC $w$= 0 on simply supported or clamped boundaries, the R-function [40–42] theory is used to find the distance function. And virtual nodes are used to implement the Derivative BC $\partial w/\partial n$ on clamped boundaries, which will be discussed in details in Section 5.2.

The outline of this paper is as follows: Section 2 presents the mechanical model of Kirchhoff plates. In Section 3, the coordinate transformation equations are summarized. Section 4 introduces the basic theory of R-function. And the details of our proposed method are presented in Section 5. The bending, free vibration and buckling problems are investigated with the proposed method in Section 6. And Section 7 summarizes our work in this paper and indicates our possible future directions.

## 2. Mechanical model of Kirchhoff plate

According to the Kirchhoff's small-deflection plate theory, the geometric equations are

$$\left.\begin{aligned}\varepsilon_x &= -z\frac{\partial^2 w(x,y)}{\partial x^2}\\ \varepsilon_y &= -z\frac{\partial^2 w(x,y)}{\partial y^2}\\ \gamma_{xy} &= -z\frac{\partial^2 w(x,y)}{\partial x\partial y}\end{aligned}\right\} \tag{1}$$

The physical equations are

$$\left.\begin{aligned}\sigma_x &= \frac{E(x,y)}{1-\nu^2}\left(\varepsilon_x+\nu\varepsilon_y\right)\\ \sigma_y &= \frac{E(x,y)}{1-\nu^2}\left(\varepsilon_y+\nu\varepsilon_x\right)\\ \tau_{xy} &= \frac{E(x,y)}{2(1+\nu)}\gamma_{xy}\end{aligned}\right\} \tag{2}$$

The bending and twisting moments are

$$M_x = -D\left(x,y\right)\left(\frac{\partial^2 w(x,y)}{\partial x^2}+\nu\frac{\partial^2 w(x,y)}{\partial y^2}\right), \tag{3}$$

$$M_y = -D\left(x,y\right)\left(\frac{\partial^2 w(x,y)}{\partial y^2}+\nu\frac{\partial^2 w(x,y)}{\partial x^2}\right), \tag{4}$$

$$M_{xy} = \int_{-h/2}^{h/2}\tau_{xy}z dz = -\left(1-\nu\right)D\left(x,y\right)\frac{\partial^2 w(x,y)}{\partial x\partial y}, \tag{5}$$

where $D = Et^3/12(1-\nu^2)$ donates the bending stiffness, $E$ and $\nu$ represent the Young's modulus and Poisson's ratio, $t$ is the thickness of the

plate.

Based on literature [1] we have the following energy equations about the bending, free vibration and buckling problem of thin plates.

For plate bending problem, we have the total potential energy of the plate in bending in the Cartesian coordinate system as follows

$$\Pi = U + W_p. \tag{6}$$

The strain energy for plates is[1].

$$U = -\frac{1}{2}\iint_A \left(M_x\frac{\partial^2 w}{\partial x^2} + M_y\frac{\partial^2 w}{\partial y^2} + 2M_{xy}\frac{\partial^2 w}{\partial x\partial y}\right)dA. \tag{7}$$

The potential energy of edge loads is not considered in current study, the potential energy of a plate under the distributed load $q(x, y)$, concentrated forces $P_i$ and moments $M_j$ can be computed as

$$W_p = -\left[\iint_A q(x,y)w dA + \sum_i P_i w_i + \sum_j P_j w_j\right], \tag{8}$$

For free vibration of thin plates, according to *Rayleigh's principle*, we have[1].

$$K_{max} = U_{max}. \tag{9}$$

Supposing that the plate is undergoing harmonic vibrations, we have the maximum kinetic energy

$$K_{max} = \frac{\omega^2}{2}\iint_A \rho t w^2(x,y)\mathrm{d}A \tag{10}$$

and the maximum strain energy

$$U_{\max} = \frac{1}{2}\iint_A D\left\{(\nabla^2 w)^2 + 2\left(1-\nu\right)\left[\left(\frac{\partial w}{\partial x\partial y}\right)^2 - \frac{\partial^2 w}{\partial x^2}\frac{\partial^2 w}{\partial y^2}\right]\right\}\mathrm{d}A. \tag{11}$$

where $\omega$ represents the frequency, $\rho$ donates the material density.

For plate buckling problems, based on *the general energy criterion for the buckling analysis of plates*[1], the critical load parameter $\lambda_{\mathrm{cr}}$ can be determined

$$\lambda_{\mathrm{cr}} = \frac{D\iint_A \left\{\left(\frac{\partial^2 w}{\partial x^2}+\frac{\partial^2 w}{\partial y^2}\right)^2 - 2\left(1-\nu\right)\left[\frac{\partial^2 w}{\partial x^2}\frac{\partial^2 w}{\partial y^2} - \left(\frac{\partial^2 w}{\partial x\partial y}\right)^2\right]\right\}\mathrm{d}x\mathrm{d}y}{-\iint_A \left[N_x^{'}\left(\frac{\partial w}{\partial x}\right)^2 + N_y^{'}\left(\frac{\partial w}{\partial y}\right)^2 + 2N_{xy}^{'}\frac{\partial w}{\partial x}\frac{\partial w}{\partial y}\right]\mathrm{d}x\mathrm{d}y}. \tag{12}$$

where $N_x^{'}$, $N_y^{'}$ and $N_{xy}^{'}$ are the reference loads, and often set to be unity in practical calculation[34].

And the boundary conditions are as follows:

For clamped edges,

$$\Gamma_1 : \left.\begin{array}{l} w = 0 \\ \dfrac{\partial w}{\partial n} = \dfrac{\partial w}{\partial x}l + \dfrac{\partial w}{\partial y}m = 0 \end{array}\right\}, \tag{13}$$

where $\boldsymbol{n}$ donates the normal tangent direction. $l = \cos\alpha$, $m = \sin\alpha$, $\alpha$ represents the angle between the $x$-axis and $\boldsymbol{n}$.

For simply supported edges, we have

$$\Gamma_2 : w = 0 \tag{14}$$

and no constraints are required for free edges.

## 3. Coordinate transformation equations

For a coordinate transformation from $\xi\eta$ domain (the reference domain) to $xy$ domain (the physical domain), we can define the transformation relations as

$$\begin{cases} x = x(\xi,\eta) \\ y = y(\xi,\eta) \end{cases}. \tag{15}$$

Let $f(x, y)$ be a function defined in the physical domain. Based on chain rules, we can obtain the first order partial derivatives of $f(x, y)$ according to the following equation

$$\begin{cases} \dfrac{\partial f}{\partial \xi} = \dfrac{\partial f}{\partial x}\dfrac{\partial x}{\partial \xi} + \dfrac{\partial f}{\partial y}\dfrac{\partial y}{\partial \xi} \\ \dfrac{\partial f}{\partial \eta} = \dfrac{\partial f}{\partial x}\dfrac{\partial x}{\partial \eta} + \dfrac{\partial f}{\partial y}\dfrac{\partial y}{\partial \eta} \end{cases}. \tag{16}$$

Eq. (16) can be rewritten into a matrix form,

$$\mathbf{J}\mathbf{F}^{(1)} = \mathbf{P}^{(1)}. \tag{17}$$

where

$$\mathbf{F}^{(1)} = \begin{bmatrix} \dfrac{\partial f}{\partial x} \\ \dfrac{\partial f}{\partial y} \end{bmatrix}, \mathbf{J} = \begin{bmatrix} \dfrac{\partial}{\partial \xi}x & \dfrac{\partial}{\partial \xi}y \\ \dfrac{\partial}{\partial \eta}x & \dfrac{\partial}{\partial \eta}y \end{bmatrix}, \mathbf{P}^{(1)} = \begin{bmatrix} \dfrac{\partial f}{\partial \xi} \\ \dfrac{\partial f}{\partial \eta} \end{bmatrix}$$

The proposed method solves PDEs defined in a physical domain through the coordinate transformation relations. According to Eq. (17), we have

$$\mathbf{F}^{(1)} = \mathbf{J}^{-1}\mathbf{P}^{(1)}, \tag{18}$$

where $\mathbf{J}$ is the Jacobian matrix.

In a similar manner, we have the following equations

$$\mathbf{A}\mathbf{F}^{(2)} + \mathbf{B}\mathbf{F}^{(1)} = \mathbf{P}^{(2)}, \tag{19}$$

$$\mathbf{F}^{(2)} = \mathbf{A}^{-1}\left(\mathbf{P}^{(2)} - \mathbf{B}\mathbf{F}^{(1)}\right), \tag{20}$$

where

$$\mathbf{F}^{(2)} = \begin{bmatrix} \dfrac{\partial^2 f}{\partial x^2} \\ \dfrac{\partial^2 f}{\partial x\partial y} \\ \dfrac{\partial^2 f}{\partial y^2} \end{bmatrix}, \mathbf{A} = \begin{bmatrix} \left(\dfrac{\partial x}{\partial \xi}\right)^2 & 2\dfrac{\partial x}{\partial \xi}\dfrac{\partial y}{\partial \xi} & \left(\dfrac{\partial y}{\partial \xi}\right)^2 \\ \dfrac{\partial x}{\partial \eta}\dfrac{\partial x}{\partial \xi} & \dfrac{\partial x}{\partial \eta}\dfrac{\partial y}{\partial \xi} + \dfrac{\partial x}{\partial \xi}\dfrac{\partial y}{\partial \eta} & \dfrac{\partial y}{\partial \eta}\dfrac{\partial y}{\partial \xi} \\ \left(\dfrac{\partial x}{\partial \eta}\right)^2 & 2\dfrac{\partial x}{\partial \eta}\dfrac{\partial y}{\partial \eta} & \left(\dfrac{\partial y}{\partial \eta}\right)^2 \end{bmatrix}, \mathbf{P}^{(2)}$$

$$= \begin{bmatrix} \dfrac{\partial^2 f}{\partial \xi^2} \\ \dfrac{\partial^2 f}{\partial \xi\partial \eta} \\ \dfrac{\partial^2 f}{\partial \eta^2} \end{bmatrix}, \mathbf{B} = \begin{bmatrix} \dfrac{\partial^2}{\partial \xi^2}x & \dfrac{\partial^2}{\partial \xi^2}y \\ \dfrac{\partial^2}{\partial \xi\partial \eta}x & \dfrac{\partial^2}{\partial \xi\partial \eta}y \\ \dfrac{\partial^2}{\partial \eta^2}x & \dfrac{\partial^2}{\partial \eta^2}y \end{bmatrix}$$

Matrices $\mathbf{J}$, $\mathbf{A}$ and $\mathbf{B}$ are constant matrices only related to the coordinate transformation relations.

## 4. R-function theory

To construct an appropriate trial function, a distance function is necessary. In this paper, the R-function proposed by T. L. Rvachev [41] is used to construct the distance function. R-function is a series of real-valued functions whose value is positive or negative only depends on whether the variable is positive or negative. To describe a complex area $\omega_0$, the Boolean operation with R-functions can be used.

The R-conjunction function is defined as

$$x_1 \wedge_\alpha x_2 = \frac{1}{1+\alpha}\left(x_1 + x_2 - \sqrt{x_1^2 + x_2^2 - 2\alpha x_1 x_2}\right). \tag{21}$$

The R-disjunction function is defined as

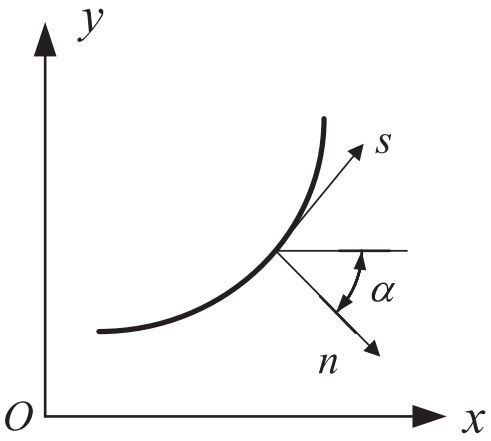



**Fig. 2.** Direction of tangent ***s*** and normal ***n***.

$$x_1 \vee_\alpha x_2 = \frac{1}{1+\alpha}\left(x_1 + x_2 + \sqrt{x_1^2 + x_2^2 - 2\alpha x_1 x_2}\right). \tag{22}$$

Choosing α = 0, and according to Eqs. (21) and (22), the following $R_0$-functions can be derived

$$f = x_1 + x_2 \pm \sqrt{x_1^2 + x_2^2}. \tag{23}$$

And we can have the following equations by differentiating Eq. (23).

$$\begin{cases} \dfrac{\partial f}{\partial x_1} = 1 \pm \dfrac{x_1}{\sqrt{x_1^2 + x_2^2}} \\ \dfrac{\partial f}{\partial x_2} = 1 \pm \dfrac{x_2}{\sqrt{x_1^2 + x_2^2}} \end{cases}. \tag{24}$$

Eq. (24) implies that the $R_0$-functions are normalized to the first order derivative when $x_1 = 0$ or $x_2 = 0$, but not differentiable at the corner points $x_1 = x_2 = 0$. The $R_0$-functions are mainly used in this paper for their smoothness, differential properties[40–42] and easier enforcement of Dirichlet boundary condition with irregular problem domains.

## 5. Improved plate deep energy method

### 5.1. Finite difference approximation and design of convolutional kernels

The proposed method divides the solution domain into several subdomains. Then the difference grids are established in each subdomain to approximate the derivatives with respect to the output of the neural

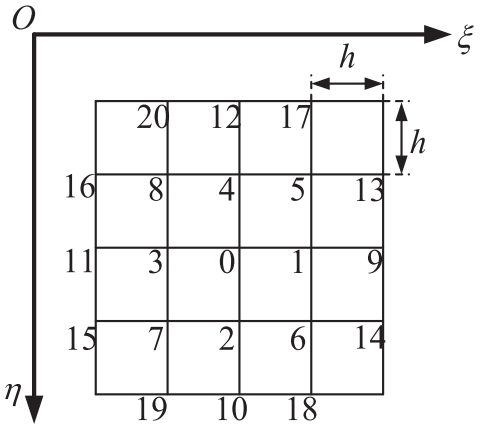



**Fig. 3.** Node number of the difference grid in the reference domain.

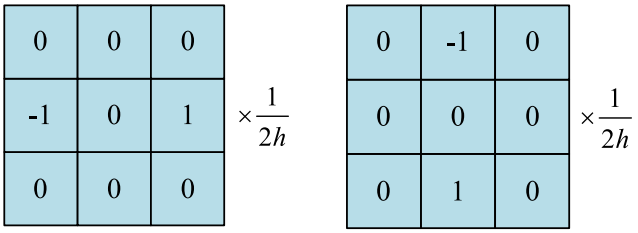



**Fig. 4.** Convolutional kernels for $(\partial f/\partial \xi)_0$(left) and $(\partial f/\partial \eta)_0$(right) at node 0.

network. In current study, a square domain $[0,1]^2$ is selected as the reference domain, and the mesh intervals $\Delta x = \Delta y = h$ (Fig. 3).

The function $f(\xi, \eta)$ is defined to be continuous in the reference domain. Based on the finite difference theory, the partial derivatives of $f(\xi, \eta)$ at node 0 (Fig. 3) are

$$\left.\begin{aligned} \left(\frac{\partial f}{\partial \xi}\right)_0 &= \frac{1}{2h}(f_1 - f_3) \\ \left(\frac{\partial f}{\partial \eta}\right)_0 &= \frac{1}{2h}(f_2 - f_4) \end{aligned}\right\}, \tag{25}$$

$$\left.\begin{aligned} \left(\frac{\partial^2 f}{\partial \xi^2}\right)_0 &= \frac{1}{h^2}(f_1 - 2f_0 + f_3) \\ \left(\frac{\partial^2 f}{\partial \eta^2}\right)_0 &= \frac{1}{h^2}(f_2 - 2f_0 + f_4) \\ \left(\frac{\partial^2 f}{\partial \xi \partial \eta}\right)_0 &= \frac{1}{4h^2}(f_6 + f_8 - f_5 - f_7) \end{aligned}\right\} \tag{26}$$

With Eqs. (25) and (26), one can design the following convolutional kernels in Figs. 4 and 5.

And the parameters of these convolutional kernels are fixed during the training process and do not need further optimization.

### 5.2. Network structure

The P-DEM[26] suggests training neural networks in a parametric (reference) domain in combination with coordinate transformation techniques, and then mapping them to the physical domain, offering a valuable approach for solving problems with irregular solution domains. Our approach differs from theirs in that we train neural networks in the physical domain and divide it into multiple subdomains. We then map each subdomain to the reference domain for derivative computation, while also applying boundary conditions on the derivatives.

In our approach, derivatives in the physical domain are obtained by transforming coordinates and calculating corresponding derivatives in the reference domain using Eqs. (18) and (20). We use pre-designed kernels with convolution operations to compute derivatives within the reference domain instead of automatic differentiation (AD). This improves computing efficiency and makes our method applicable to problems with derivative boundary conditions (DBCs).

First, we need to find the distance function $\phi(x, \quad y)$ based on the physical domain. Then, convolution operations are used to calculate the energy equation and loss function without requiring gradients of the input data. AD is only used for backpropagation during loss calculation. As shown in Fig. 6, a fully connected network is used as the main structure in the proposed method, which we believe is applicable to most mechanical problems. The solution steps of our method are as follows:

**Step (1)** Domain decomposition and determination of coordinate transformation relations. The proposed method initially decomposes the solution domain into several subdomains. For each subdomain, the coordinate transformation relations between the reference

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & -2 & 1 \\ 0 & 0 & 0 \end{bmatrix} \times \frac{1}{h^2} \quad \begin{bmatrix} 0 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 1 & 0 \end{bmatrix} \times \frac{1}{h^2} \quad \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix} \times \frac{1}{4h^2}$$

**Fig. 5.** Convolutional kernels for $\left(\partial^2 f/\partial\xi^2\right)_0$(left), $\left(\partial^2 f/\partial\eta^2\right)_0$ (middle) and $\left(\partial^2 f/\partial\xi\partial\eta\right)_0$ (right) at node 0.

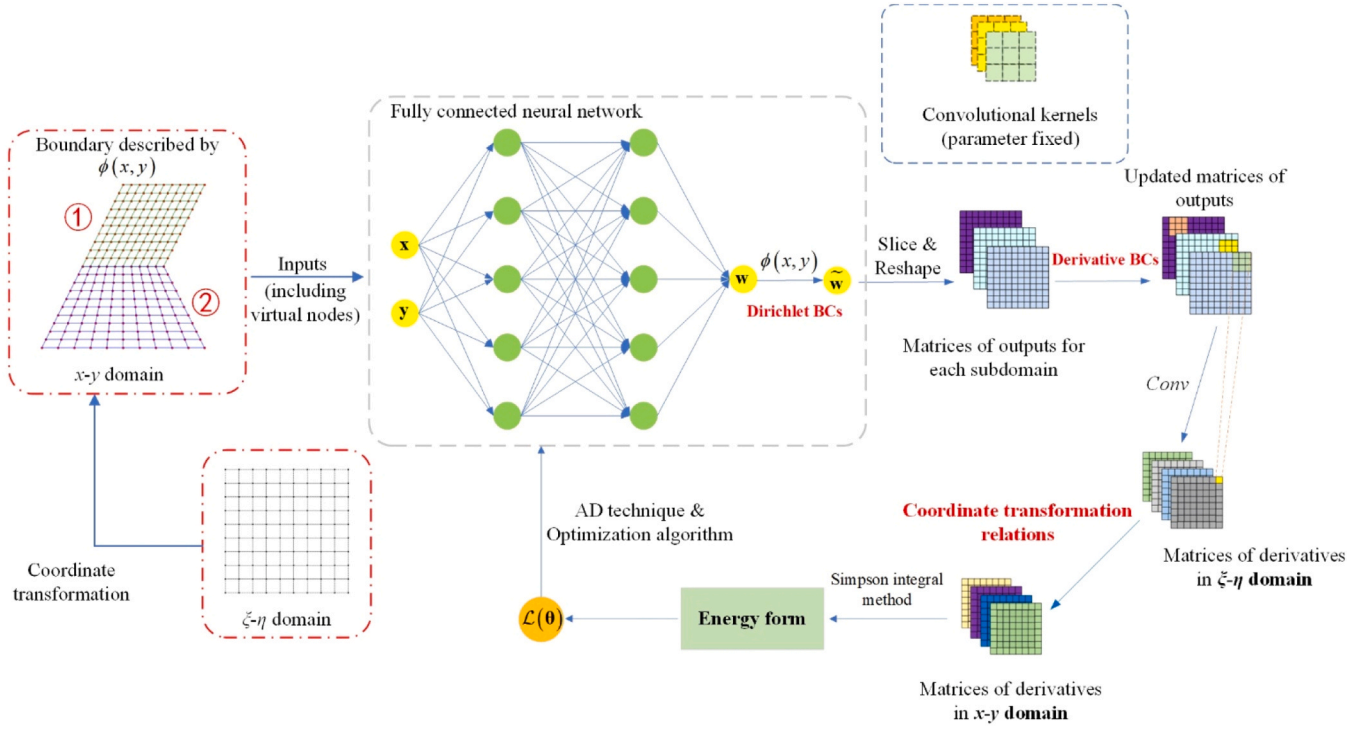



**Fig. 6.** Schematic diagram of solution steps of the proposed method.

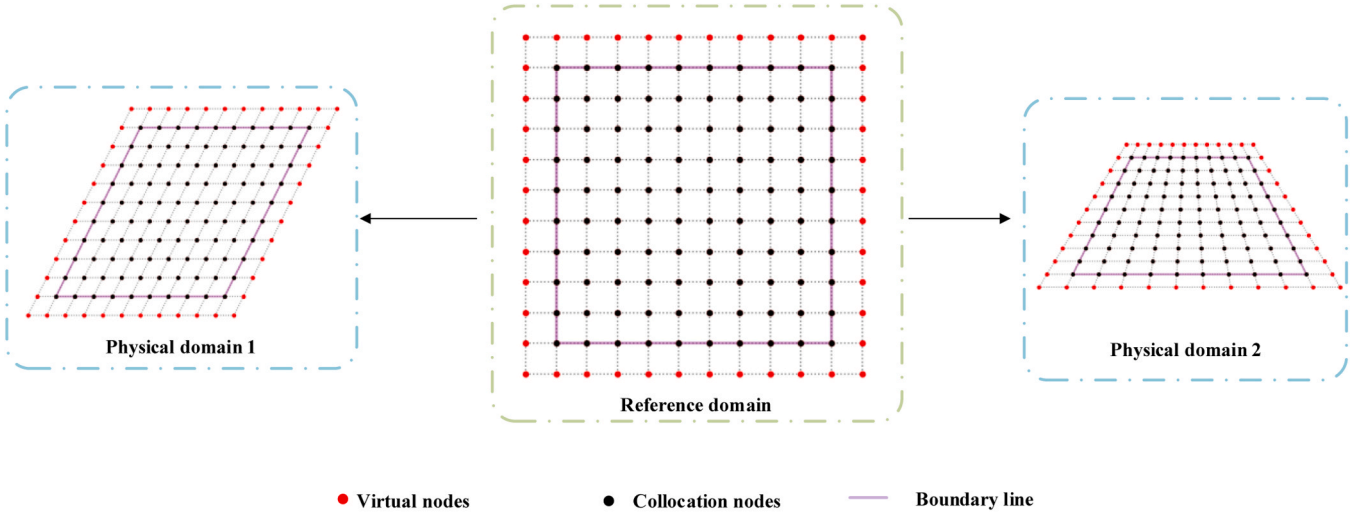



**Fig. 7.** Difference grid in reference domain and its projections to physical domains.

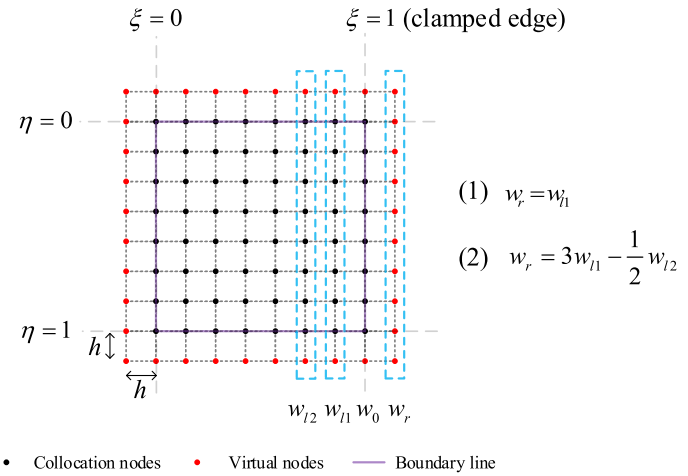



**Fig. 8.** Diagram of the enforcement of clamped boundary conditions at edge $\xi = 1$ within the reference domain.

domain and the physical domain are obtained through interpolation techniques. Before training, the training nodes are generated uniformly in the reference domain and transformed to the physical domain to obtain the input *xy* data. And the input data remain fixed during the training process in order to cooperate with the finite difference and integral operation.

**Step (2)** Data generation. The input data for the fully connected neural network is a sequence of nodal coordinates from each subdomain. And the coordinates of the virtual nodes should be included in the input data. Assuming the output of the network for a subdomain constitutes a deflection vector, for the convenience of convolutional operations, we reshape the vectors to be represented in the form of a matrix (reshaped deflection matrix). Let the size of the reshaped deflection matrix be ($L$, $M$), since the convolutional kernels are designed in a matrix size of which is 3 × 3. After the convolution operation, the size of the result matrix shrinks to ($L$-2, $M$-2). If no additional paddings are provided in the deflection matrix, the derivatives at the boundaries are lost, causing problems with the integral in the energy equation. To avoid this problem, virtual nodes should be considered when establishing difference grid in the reference domain (Fig. 7). The size of the reshaped deflection matrix will then be ($L$+2, $M$+2). After that, under the coordinate transformation relations determined in Step (1), the constant matrices mentioned in Section 3 are calculated for each nodal point and stored for further usage.

**Step (3)** Multiplying the output of neural network and the distance function $\phi(x, \quad y)$ enables the satisfaction of the Dirichlet boundary conditions.

**Step (4)** Implementing derivative boundary conditions using virtual nodes. In this step, we focus solely on the clamped boundary conditions in the context of Kirchhoff plate problems. As difference grids are utilized, the proposed method can effectively enforce boundary conditions within the reference domain. For clamped edges, the Dirichlet boundary condition $w$= 0 is satisfied in Step (3).

Take edge $\xi = 1$ for example, $w$= 0 results in $\frac{\partial w}{\partial \eta} = 0$, according to Eq. (17), we have

$$\begin{bmatrix} \frac{\partial w}{\partial x} \\ \frac{\partial w}{\partial y} \end{bmatrix} = \mathbf{J}^{-1} \begin{bmatrix} \frac{\partial w}{\partial \xi} \\ 0 \end{bmatrix}. \tag{27}$$

Setting the following notation

$$\mathbf{J}^{-1} = \begin{bmatrix} J_{11} & J_{12} \\ J_{21} & J_{22} \end{bmatrix}, \tag{28}$$

and we have

$$\left.\begin{aligned} \frac{\partial w}{\partial x} &= J_{11}\frac{\partial w}{\partial \xi} \\ \frac{\partial w}{\partial y} &= J_{21}\frac{\partial w}{\partial \xi} \end{aligned}\right\} \tag{29}$$

Substituting Eq. (29) into Eq. (13) yields

$$\theta_n = \frac{\partial w}{\partial n} = (lJ_{11} + mJ_{21})\frac{\partial w}{\partial \xi} = 0. \tag{30}$$

For a coordinate transformation with zero-strangeness, Eq. (30) will result in

$$\frac{\partial w}{\partial \xi} = 0. \tag{31}$$

Eq. (31) can be satisfied approximately by setting the nodal value of virtual nodes

$$w_r = w_{l1} \tag{32}$$

According to Eq. (25), we have

$$\left(\frac{\partial f}{\partial \xi}\right)_{\xi=1} \approx \frac{1}{2h}(w_{l1} - w_r) = 0. \tag{33}$$

Along edge ξ = 1, we can substitute the deflections $w_r$ of the right-side virtual nodes with the $w_l$ of the left-side virtual nodes (as shown in Fig. 8). This substitution can be performed directly with the assistance of AD (Automatic Differentiation) techniques.

High-order difference schemes can be used to obtain precise approximations. Assume that the deflection $w$ is a cubic function in direction $w_{l2}$-$w_{l1}$-$w_0$-$w_r$, then

$$w = A + B\left(\frac{\zeta}{h}\right) + C\left(\frac{\zeta}{h}\right)^2 + D\left(\frac{\zeta}{h}\right)^3. \tag{34}$$

We have

$$\left.\begin{aligned} &(w)_{\zeta=0} = w_0 = 0 \\ &\left(\frac{\partial w}{\partial \zeta}\right)_{\zeta=0} = \left(\frac{\partial w}{\partial \zeta}\right)_{\zeta=1} = 0 \\ &(w)_{\zeta=-h} = w_{l1} \\ &(w)_{\zeta=-2h} = w_{l2} \end{aligned}\right\} \tag{35}$$

Substituting Eqs. (34) into Eq. (35), the coefficients in Eq. (34) are obtained as

$$A = 0, B = 0, C = 2w_{l1} - \frac{w_{l2}}{4}, D = w_{l1} - \frac{w_{l2}}{4}. \tag{36}$$

The substitution of Eq. (36) into Eq. (34) yields

$$w = \frac{\zeta^2}{h^2}\left(2w_{l1} - \frac{w_{l2}}{4}\right) + \frac{\zeta^3}{h^3}\left(w_{l1} - \frac{w_{l2}}{4}\right). \tag{37}$$

And we have the following deflection expression about virtual nodes

$$w_r = (w)_{\zeta=h} = 3w_{l1} - \frac{1}{2}w_{l2}. \tag{38}$$

For the enforcement of clamped boundary conditions, Eqs. (32) and (38) are both feasible, one just needs to set the deflection value of virtual nodes through variable substitution, which can be traced by AD technique and put into the computation graph[48] of loss. The deflection of other virtual nodes near a clamped edge can also be derived in a similar manner.

**Step (5)** Calculating derivatives in the physical domain involves an initial computation of derivatives in the $\xi$-$\eta$ domain (the reference domain) using the difference method and convolution operations. Subsequently, the constant matrices computed in step (2) are utilized to derive derivatives in the $x$-$y$ domain (the physical domain) based on Eqs. (18) and (20).

**Step (6)** Calculating the loss function according to the type of problems. The loss function is computed based on the type of problem. And the details will be discussed in Section 5.3.

**Step (7)** Using optimization algorithms, the parameters of the neural network are updated continuously through iterations until either convergence is achieved or the maximum number of iterations is reached. The normalized form of the termination criterion is represented as follows:

$$\text{conv} = \frac{\sum_{i}^{\overline{N}} \left(W_i^* - W_i\right)^2}{\sum_{i}^{\overline{N}} \left(W_i^*\right)^2} \leq \varepsilon, \tag{39}$$

where $\varepsilon = 10^{-6}$, $\overline{N}$ is the total number of the training nodes, $W$ and $W^*$ represent the output of neural network in current and previous iteration, respectively.

### 5.3. Loss function

Defining the output of the presented method as

$$W(\mathbf{x};\boldsymbol{\theta}) = \phi(\mathbf{x})\mathcal{N}(\mathbf{x};\boldsymbol{\theta}), \tag{40}$$

where vector $\mathbf{x} = (x, y)$ represents the input, $\phi(\mathbf{x})$ is the distance function which is constructed according to the Dirichlet boundary conditions in Eqs. (13) and (14), $\mathcal{N}(\mathbf{x};\boldsymbol{\theta})$ is the output of neural network. The remaining boundary conditions of clamped edges are enforced through approaches in step (4) of the solution steps. We have the loss functions in Section 4.4.1 without terms regarding boundary conditions.

The energy method is adopted in current study. Supposing that the solution domain is decomposed into $N$ subdomains, the superposition of the energy of each subdomain yields the total energy of a plate system.

#### 5.3.1. Loss function for bending problems

Without consideration of boundary conditions, the loss function for bending analysis is

$$L_{\text{bending}} = \sum_{i=1}^{N} \Pi_i(\mathbf{x}_i;\boldsymbol{\theta}), \tag{41}$$

where the total energy equation of the $i$-th subdomain is defined as

$$\Pi_i(\mathbf{x};\boldsymbol{\theta}) = \frac{1}{2}\int_{A_i} \mathbf{k}_i^{\mathrm{T}}\mathbf{D}\mathbf{k}_i \mathrm{d}A - \int_{A_i} qW(\mathbf{x}_i;\boldsymbol{\theta})\mathrm{d}A, \tag{42}$$

and $q$ represents the transverse load. $w$ is the deflection of plate.

The curvature vector $\mathbf{k}$ is defined as

$$\mathbf{k} = \begin{bmatrix} k_x & k_y & k_{xy} \end{bmatrix}^{\mathrm{T}} = -\left[\frac{\partial^2 W(\mathbf{x};\boldsymbol{\theta})}{\partial x^2} \quad \frac{\partial^2 W(\mathbf{x};\boldsymbol{\theta})}{\partial y^2} \quad 2\frac{\partial^2 W(\mathbf{x};\boldsymbol{\theta})}{\partial x \partial y}\right]^{\mathrm{T}}. \tag{43}$$

And the bending stiffness matrix

$$\mathbf{D} = \frac{Et^3}{12(1-\nu^2)}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1-\nu)/2 \end{bmatrix} = D\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1-\nu)/2 \end{bmatrix}, \tag{44}$$

where $t$ is the thickness of plate, $E$ donates the Young's modulus, $\nu$ is the Poisson's ratio.

#### 5.3.2. Loss function for free vibration and buckling

The initial research into applying deep learning techniques to solve the problems of free vibration and buckling of thin plates was conducted by Zhuang et al.[34]. Our study focuses on eliminating boundary losses and improving the training efficiency. In this paper, we consider that the plate is undergoing harmonic vibrations, with the fundamental frequency being our primary concern. *Rayleigh's principle* is utilized to determine the lowest natural frequency. After eliminating the boundary losses, the loss function for free vibration analysis is defined as[34]:

$$L_{\text{vibration}} = \omega^2(\mathbf{x};\boldsymbol{\theta}), \tag{45}$$

where

$$\omega^2(\mathbf{x};\boldsymbol{\theta}) = \frac{2\sum_{i}^{N} U_i^{\max}}{\sum_{i}^{N} \iint_{A_i} \rho h W^2(\mathbf{x}_i;\boldsymbol{\theta})\mathrm{d}x\mathrm{d}y} \tag{46}$$

with

$$U_{\max} = \frac{1}{2}\iint_A D\left\{\left(\nabla^2 W(\mathbf{x};\boldsymbol{\theta})\right)^2 + 2(1-\nu)\left[\left(\frac{\partial^2 W(\mathbf{x};\boldsymbol{\theta})}{\partial x \partial y}\right)^2 - \frac{\partial^2 W(\mathbf{x};\boldsymbol{\theta})}{\partial x^2}\frac{\partial^2 W(\mathbf{x};\boldsymbol{\theta})}{\partial y^2}\right]\right\}\mathrm{d}x\mathrm{d}y. \tag{47}$$

With *the general energy criterion for the buckling analysis of plates*, the loss function for plate buckling problem is defined as[34].

$$L_{\text{buckling}} = \lambda(\mathbf{x};\boldsymbol{\theta}), \tag{48}$$

$$\lambda(\mathbf{x};\boldsymbol{\theta}) = \frac{\frac{1}{2}\sum_{i}^{N}\iint_{A_i} D\left\{\left(\frac{\partial^2 W(\mathbf{x}_i;\boldsymbol{\theta})}{\partial x^2}+\frac{\partial^2 W(\mathbf{x}_i;\boldsymbol{\theta})}{\partial y^2}\right)^2 + 2(1-\nu)\left[\left(\frac{\partial^2 W(\mathbf{x}_i;\boldsymbol{\theta})}{\partial x\partial y}\right)^2 - \frac{\partial^2 W(\mathbf{x}_i;\boldsymbol{\theta})}{\partial x^2}\frac{\partial^2 W(\mathbf{x}_i;\boldsymbol{\theta})}{\partial y^2}\right]\right\}\mathrm{d}x\mathrm{d}y}{-\frac{1}{2}\sum_{i}^{N}\iint_{A_i}\left[N_x\left(\frac{\partial W(\mathbf{x}_i;\boldsymbol{\theta})}{\partial x}\right)^2 + N_y\left(\frac{\partial W(\mathbf{x}_i;\boldsymbol{\theta})}{\partial y}\right)^2 + 2N_{xy}\frac{\partial W(\mathbf{x}_i;\boldsymbol{\theta})}{\partial x}\frac{\partial W(\mathbf{x}_i;\boldsymbol{\theta})}{\partial y}\right]\mathrm{d}x\mathrm{d}y} \tag{49}$$

where

#### 5.3.3. Node discretization and Simpson integral in reference domain

The aforementioned loss functions are defined in the physical domain. We transform these functions into the reference domains. A square domain is selected as the reference domain for all cases. The uniformly distributed nodes are used. To calculate the integrals in Eqs. (42), (45) and (48), the Simpson integral formula is employed since it adapts to the discretization scheme of collocation points in this paper. The one-dimensional Simpson integral is

$$\int_a^b f(x)dx \approx \frac{\Delta x}{6}\left[f(a)+4\sum_{k=0}^{N-1}f(x_{2k+1})+2\sum_{k=1}^{N}f(x_{2k})+f(b)\right]. \tag{50}$$

And the following two-dimensional integration coefficient matrix can be obtained as

$$\mathbf{W}^{\text{r}}_{\text{simpson}} = \frac{\Delta x\Delta y}{36}\begin{pmatrix} 1 & 4 & 2 & 4 & \cdots & 2 & 4 & 1 \\ 4 & 16 & 8 & 16 & \cdots & 8 & 16 & 4 \\ 2 & 8 & 4 & 8 & \cdots & 4 & 8 & 2 \\ 4 & 16 & 8 & 16 & \cdots & 8 & 16 & 4 \\ \vdots & \vdots & \vdots & \vdots & \cdots & \vdots & \vdots & \vdots \\ 2 & 8 & 4 & 8 & \cdots & 4 & 8 & 2 \\ 4 & 16 & 8 & 16 & \cdots & 8 & 16 & 4 \\ 1 & 4 & 2 & 4 & \cdots & 2 & 4 & 1 \end{pmatrix}. \tag{51}$$

It should be noted that the integration coefficient matrix in Eq.(51) is based on the reference domain. To compute the integration coefficient matrix in the physical domain, we have

$$\mathbf{W}^{\text{p}}_{\text{simpson}} = \boldsymbol{\Phi}\odot\mathbf{W}^{\text{r}}_{\text{simpson}}, \tag{52}$$

where $\odot$ represents the Hadamard product. $\boldsymbol{\Phi}$ is calculated by determinant's value from the Jacobian matrix $\mathbf{J}$ in Eq. (17) of the training nodes

$$\boldsymbol{\Phi} = \begin{bmatrix} |\mathbf{J}_{11}| & |\mathbf{J}_{12}| & \cdots & |\mathbf{J}_{1m}| \\ |\mathbf{J}_{21}| & |\mathbf{J}_{22}| & \cdots & |\mathbf{J}_{2m}| \\ \vdots & \vdots & \vdots & \vdots \\ |\mathbf{J}_{n1}| & |\mathbf{J}_{n2}| & \cdots & |\mathbf{J}_{nm}| \end{bmatrix}. \tag{53}$$

And $n$ and $m$ donate the nodal location of rows and columns in the reference domain, respectively.

**Table 1**
Activation function, layers and neurons used in each numerical example.

| Problem | | Activation function | ($N_{\text{l}}$, $N_{\text{n}}$) |
|---|---|---|---|
| Bending | Irregular plate | $x^2$ | * |
| | Triangular plate | $tanh(x)$ | (4, 30) |
| | L-shaped plate | $x^2$ | (5, 30) |
| Free vibration | Rectangular plates | $tanh(x)$ | * |
| | Annular plates | $x^2$ | * |
| Buckling | Skew plates | $tanh(x)$ | (5, 20) |
| | Irregular plates | $x^2$ | (5, 20) |

## 6. Numerical examples

### 6.1. Hyperparameter settings

The neural network approaches have more hyperparameters than the usual numerical methods, and the majority of them are still empirical. The hyperparameters adopted in the current study are introduced in this section before our numerical experiments.

(a) Learning rates. The choice of learning rate has a direct impact on training results and efficiency. A greater learning rate is typically necessary in the early stages of training to speed network convergence, whereas a lower learning rate is typically required in the later stages of training to fine-tune the network model. In the current study, the learning rate is suggested as follows.

$$\alpha_n = \begin{cases} 1\times10^{-3}, & n \le 2000 \\ 5\times10^{-4}, & 2000 < n \le 4000 \\ 1\times10^{-4}, & 4000 < n \le 6000 \\ 5\times10^{-5}, & 6000 < n \end{cases}, \tag{54}$$

where $n$ represents the number of iterations.

(b) Activation functions, layers and neurons.

Table 1 gives the configuration of activation function, the number of hidden layers ($N_{\text{l}}$), and the number of neurons in each hidden layer ($N_{\text{n}}$) for each numerical example. The "* " mark in the table indicates that we will discuss the impact of the values of $N_{\text{l}}$ and $N_{\text{n}}$ on the computational results in the corresponding examples.

In our study, two activation functions are used. One is the *tanh* function, which is a preset activation function in most deep learning frameworks, such as Pytorch, Tensorflow, etc. The other one is the quadratic activation function with a simple form of $x^2$. When applying this activation function, the network's input data cannot be too vast, thus a normalization input is necessary.

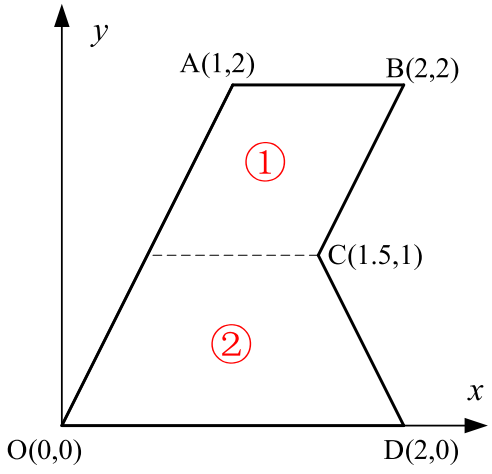



**Fig. 9.** Irregular plate.

**Table 2**
Effect of training nodes and hidden layer neuron count on accuracy and computational efficiency with fixed hidden layer depth.

| $N_{int}$ | $N_l$ | $N_n$ | $w_c$ ($\times 10^{-3}$/m) | $e_1$ | $e_2$ | Time (sec) | Memory (MiB) |
|---|---|---|---|---|---|---|---|
| 21 × 21 × 2 | 5 | 20 | -5.9857 | -3.32% | $1.04 \times 10^{-8}$ | 36.57 | 177.12 |
| | 5 | 30 | -6.0136 | -2.84% | $9.56 \times 10^{-9}$ | 28.62 | 179.38 |
| | 5 | 40 | -5.9397 | -4.12% | $3.24 \times 10^{-8}$ | 48.31 | 183.04 |
| 29 × 29 × 2 | 5 | 20 | -6.1547 | -0.49% | $7.81 \times 10^{-10}$ | 18.07 | 177.89 |
| | 5 | 30 | -6.1358 | -0.80% | $2.11 \times 10^{-9}$ | 38.01 | 179.91 |
| | 5 | 40 | -6.1997 | 0.24% | $5.29 \times 10^{-10}$ | 26.08 | 182.21 |
| 35 × 35 × 2 | 5 | 20 | -6.1595 | -0.41% | $5.89 \times 10^{-10}$ | 18.57 | 177.90 |
| | 5 | 30 | -6.1937 | 0.15% | $6.59 \times 10^{-10}$ | 24.88 | 179.74 |
| | 5 | 40 | -6.1770 | -0.12% | $5.81 \times 10^{-10}$ | 20.79 | 182.15 |
| 41 × 41 × 2 | 5 | 20 | -6.1223 | -1.02% | $1.16 \times 10^{-9}$ | 17.60 | 177.07 |
| | 5 | 30 | -6.1633 | -0.35% | $6.83 \times 10^{-10}$ | 25.11 | 179.98 |
| | 5 | 40 | -6.1892 | 0.07% | $5.17 \times 10^{-10}$ | 26.29 | 182.78 |

**Table 3**
Accuracy, time, and memory for different network configurations under 41×41×2 training nodes.

| $N_l$ | $N_n$ | $w_c$ ($\times 10^{-3}$/m) | $e_1$ | $e_2$ | Time (sec) | Memory (MiB) |
|---|---|---|---|---|---|---|
| 2 | 20 | -6.1700 | -28.22% | $7.50 \times 10^{-7}$ | 38.58 | 175.57 |
| 3 | 20 | -6.1633 | -5.61% | $4.32 \times 10^{-8}$ | 30.86 | 175.96 |
| 4 | 20 | -3.8178 | -0.19% | $2.30 \times 10^{-9}$ | 32.42 | 177.47 |
| 5 | 20 | -4.8236 | -1.02% | $1.16 \times 10^{-9}$ | 17.60 | 177.07 |
| 2 | 30 | -6.1892 | -7.82% | $3.91 \times 10^{-8}$ | 29.53 | 177.18 |
| 3 | 30 | -4.5553 | -6.82% | $3.61 \times 10^{-8}$ | 18.85 | 178.28 |
| 4 | 30 | -5.7358 | -0.24% | $5.17 \times 10^{-10}$ | 19.96 | 178.71 |
| 5 | 30 | -5.7899 | -0.35% | $6.83 \times 10^{-10}$ | 25.11 | 179.98 |
| 2 | 40 | -5.0424 | -5.28% | $1.77 \times 10^{-8}$ | 13.65 | 179.68 |
| 3 | 40 | -5.8746 | -1.58% | $5.14 \times 10^{-9}$ | 33.79 | 180.96 |
| 4 | 40 | -6.0885 | -0.08% | $5.69 \times 10^{-10}$ | 41.35 | 180.53 |
| 5 | 40 | -6.1795 | 0.07% | $5.17 \times 10^{-10}$ | 26.29 | 182.78 |

(c) The Adam [46] optimization algorithm is selected for all numerical experiments. Xavier uniform distribution [47] is selected for initialization of network parameters. Without any special notification, current study limits the number of iterations to $N = 6000$.

### 6.2. Bending of irregular plate

As shown in Fig. 9, we consider a Kirchhoff plate of irregular shape with mixed boundary conditions. The edges OA, AB, OD are simply supported, and BC and CD are clamped. The material properties are: Elastic modulus $E = 10920000$ Pa, Poisson's ratio $\nu = 0.3$. And the thickness $t = 0.01$ m. The plate is subjected to a uniform load $q_0 = 1$ N/m$^2$. In this example, we enforce the boundary conditions with the approaches mentioned in Section 5.2.

We use R-function to construct the distance function. According to the geometry, we have

$$\left.\begin{aligned} \omega_1 &= y(y-2) \geq 0 \\ \omega_2 &= 2x - y \geq 0 \\ \omega_3 &= y - 2(x-1) \geq 0 \\ \omega_4 &= -y - 2(x-2) \geq 0 \end{aligned}\right\}$$

The distance function is

$$\phi = (\omega_1 \wedge \omega_2) \wedge (\omega_3 \vee \omega_4).$$

We set up two subdomains (Fig. 9), and the derivative boundary conditions of edges BC and CD are implemented based on the difference scheme in the reference domain. The coordinate transformation relations are considered in each subdomain.

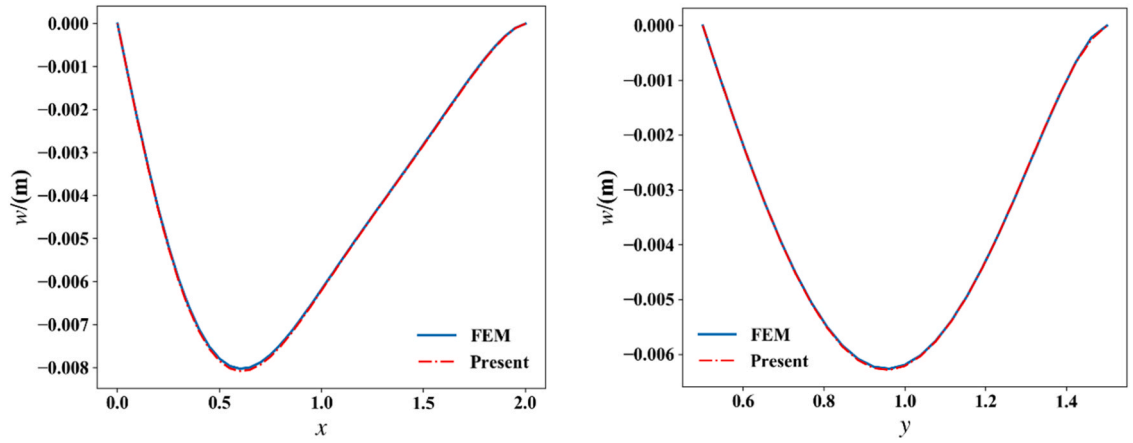
To depict the convergence behavior, we set a measurement point c (1,1) to measure the relative errors between the proposed method and FEM results given by ABAQUS using 4068 STRI3 elements. And the relative errors are given by

$$e_1 = \frac{(w_c - w_c^*)}{w_c} \tag{55}$$

Where $w_c$ and $w_c^*$ are deflections at point c given by present method and FEM, respectively.

The following equation (mean squared error) is used to measure the global errors

$$e_2 = \frac{1}{n}\sum_{i}^{n}\left(w_i - w_i^*\right)^2 \tag{56}$$




(a) Deflections along axis $x$=1m. (b) Deflections along axis $y$=1m.

**Fig. 10.** Comparison of deflections given by the presented method($N_l$=5, $N_n$ =30, 41 ×41 ×2 training nodes) and the FEM.

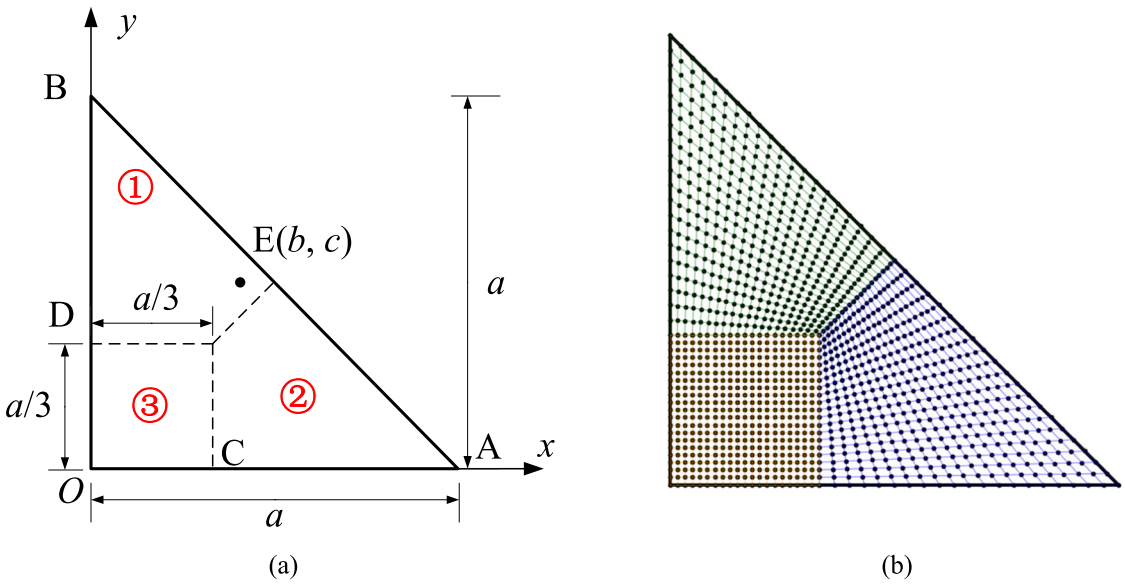



**Fig. 11.** Triangular plate (a) and difference grids (b).

**Table 4**
Maximum dimensionless deflections with different load points under simply supported BC.

| (b, c) | Present | Theory | Relative errors |
|---|---|---|---|
| (a/3, a/3) | 4.2767 | 4.2677 | 0.21% |
| (a/2, a/3) | 2.5544 | 2.5487 | 0.22% |
| (a/6, a/3) | 3.4870 | 3.4760 | 0.31% |
| (a/2, a/6) | 2.8668 | 2.8684 | -0.06% |
| (a/6, a/6) | 2.6532 | 2.6583 | -0.19% |

Where $n$ donates the total number of measurement points, and these points are taken from the mesh nodes in FEM. And $w_i$ and $w_i^*$ are deflections given by present method and FEM at $i$-th measurement point, respectively.

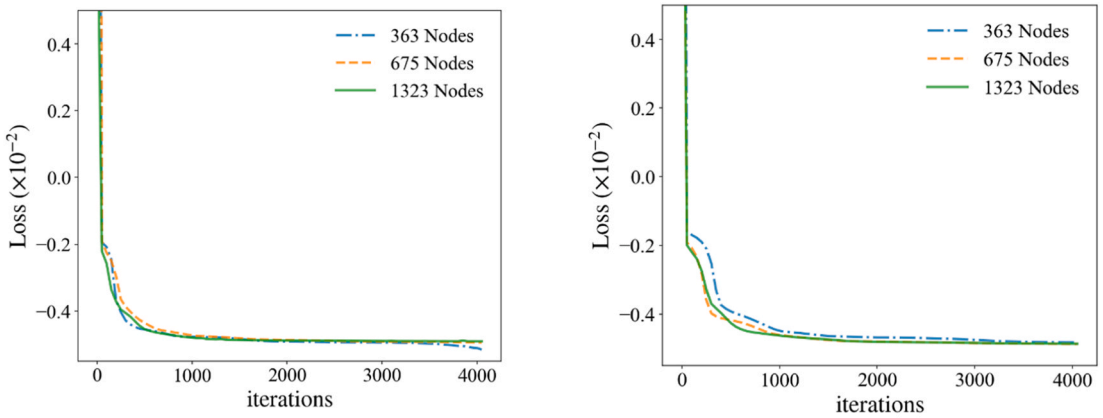
**Table 5**
Maximum dimensionless deflections with different load points under mixed BC.

| (b, c) | Present | FEM | Relative errors |
|---|---|---|---|
| (a/3, a/3) | 3.0574 | 3.0456 | 0.38% |
| (a/2, a/3) | 1.9151 | 1.9124 | 0.14% |
| (a/6, a/3) | 2.0227 | 2.0101 | 0.62% |
| (a/2, a/6) | 1.5646 | 1.5616 | 0.19% |
| (a/6, a/6) | 1.8824 | 1.8500 | 1.72% |

We examined this numerical example further to demonstrate the performance of the suggested method by modifying the number of integration points ($N_{\text{int}}$), $N_{\text{l}}$, and $N_{\text{n}}$ while keeping the other parameters constant.The results are provided in Tables 2 and 3, along with the required computation time and memory. Increasing integration points enhances the efficiency of our method across different network architectures, as shown in Table 2, for a configuration with $41 \times 41 \times 2$ integration points, both networks achieve high accuracy. Table 3 explores the impact of increasing hidden layers with this integration setting. With the increase in the number of training points, the memory requirements of the network remain stable. Adding layers and neurons significantly improves the accuracy. The right network structure and integration settings ensure quick convergence, saving computational time.

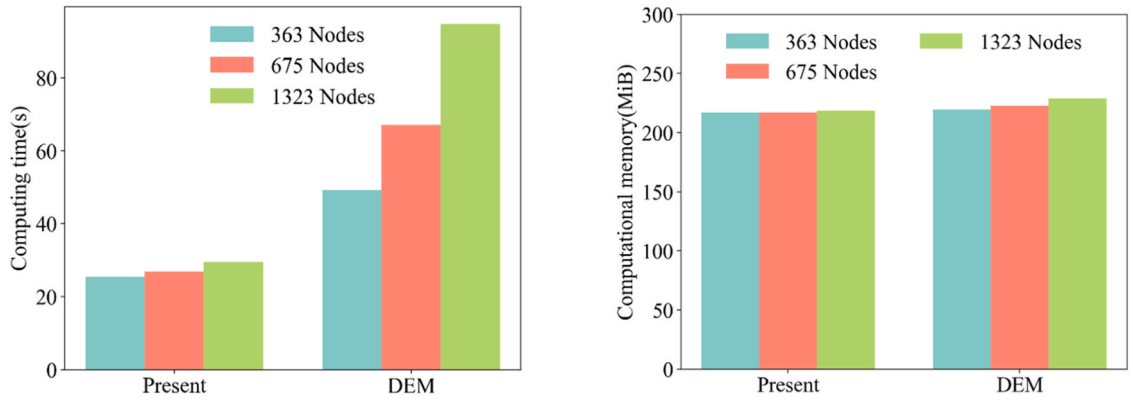
It should be noted that the initialization of network parameters can also impact the training, but that is not the focus of this article.

Fig. 10 shows that the deflections obtained by both methods fit each other well. The deflections near the boundary are also stable and consistent with those given by ABAQUS (FEM). From these data, it can be observed that with a minimum of 4 hidden layers, the neuron configurations mentioned above can achieve high accuracy. This suggests that a network depth of 4–5 layers is sufficient. For computational



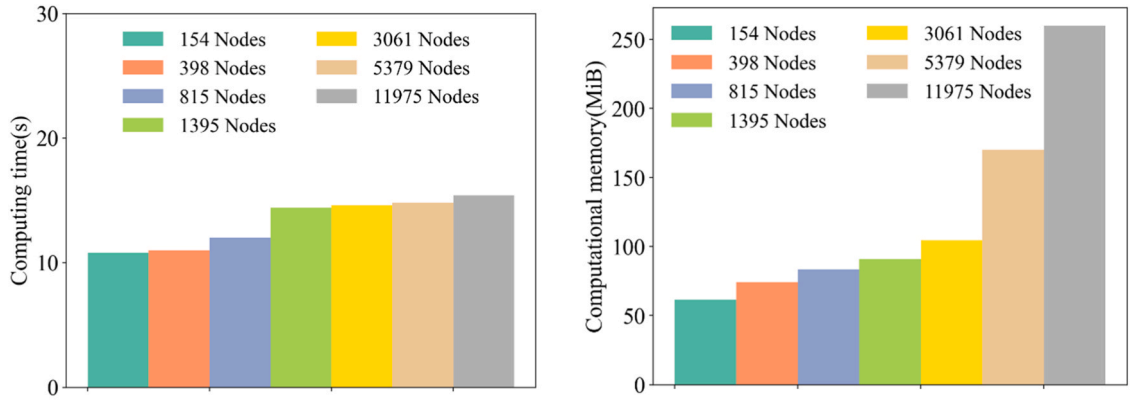

(a) Loss convergence history of the presented method (a) Loss convergence history of the DEM

**Fig. 12.** Comparison of convergence histories between the presented method and DEM.




(a) Computing time (CPU). (b) Required computation memory.

**Fig. 13.** Comparison of computation time and memory required by the presented method and DEM under the maximum iteration.




(a) Computing time (CPU). (b) Required computation memory.

**Fig. 14.** Time and memory required in finite element analysis run by ABAQUS.

efficiency considerations, the choice of 20–40 neurons per hidden layer in this paper also yields satisfactory results.

### 6.3. *Bending of triangular plate with mixed boundary conditions*

We consider an isosceles right triangular plate (Fig. 11.a) with length $a$= 1.5 m and thickness $t = 0.01$ m under a concentrated force $P_0 = 1$ N perpendicular to the $x$-$y$ plane at point E($b$, $c$). Young's modulus $E = 10920000$ Pa, Poisson's ratio $\nu = 0.3$. Two boundary conditions (BCs) are considered: (1) Simply supported BC, all edges of the plate are simply supported. (2). Mixed BC, edges OD, OC and AB are simply supported while DB and CA are clamped. To apply the mixed boundary conditions, the solution domain is divided into 3 subdomains and we set up difference grid for each of them (Fig. 11.b), $3 \times 21 \times 21$ collocation nodes are used. It should be noted that virtual nodes are not shown in the figure for simplicity. The distance function can be easily constructed as:

$$\phi = xy(y + x - a).$$

With dimensionless formula $\overline{w} = 10^3 Dw/(P_0 a^2)$, by changing the location of point E, the maximum dimensionless deflection given by the proposed method, the literature [1] and the FEM are listed in Tables 4 and 5. The finite element analysis is performed in ABAQUS software, using 5379 STRI3 elements. Good agreement among the results can be observed. The effectiveness of the present method with mixed boundary conditions has also been verified.

A comparison study with the DEM to analyze the problem under boundary condition (1) and point E($a$/3, $a$/3) is also carried out. Both methods share the same hyperparameters and the same distance function to eliminate the boundary losses, and the maximum iteration number $N$ is set to be 4000. In Fig. 12, the loss convergence history of the proposed method agrees well with that of the DEM, indicating that using the finite difference to approximate the loss function is viable. As illustrated in Fig. 13, we have recorded the computation time and memory consumption of the two methods during the training process with different number of training nodes. It can be found that the required training memory is stable and close for both methods, while less computation time is required for the proposed method, which demonstrates the remarkable training efficiency. When using neural network-based methods to solve partial differential equations, there are typically two parts involving derivatives:(1). Derivatives of network outputs with respect to inputs. (2). During backpropagation, the derivatives of the loss function with respect to network parameters. Our paper primarily focuses on addressing part (1). The original DEM computes these derivatives using automatic differentiation which is based on computational graph techniques[48], while we employ numerical differentiation (ND) methods. The advantage of our approach is that there is no need to construct computational graphs for the derivative calculations in part (1). Consequently, during the forward propagation of the network, there is no requirement to track variables related to the network inputs (such as $x$ and $y$). At the same time, both our method and DEM rely on automatic differentiation to compute the derivatives related to part (2). The difference is that we construct an approximate loss function using numerical differentiation techniques, which results

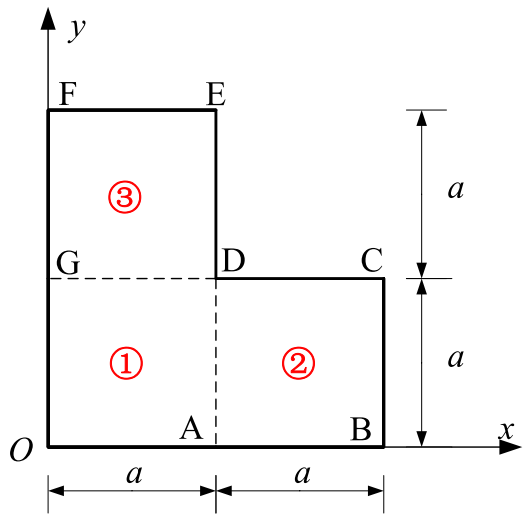



**Fig. 15.** L-shaped plate.

in a simpler computational graph. This simplification improves the efficiency of the loss backpropagation.

Fig. 14 illustrates the amount of time and memory needed for the finite element solution procedure. From the comparative data in Figs. 13 and 14, it is evident that there still exists a significant gap between our approach and finite element analysis in terms of solving time and required memory. The primary reason for this disparity lies in the fact that the neural network-based approach is founded on optimization principles, necessitating iterative updates of network parameters to minimize the loss function. In contrast, methods such as finite element analysis only require the solution of stiffness equations once in linear problem-solving. However, the neural network-based approach describes the displacement field over the entire solution domain using neural networks, eliminating the need to assemble the global stiffness matrix. Nodes within the solution domain serve as training data for the network, and increasing the number of nodes does not impose a significant memory burden during the training process. From another perspective, addressing global solutions to differential equations within irregular solution domains presents a substantial challenge. Current study leverages the robust fitting capabilities of neural networks, providing an approach for obtaining globally differentiable solutions to PDEs within irregular domains. Different methods offer their unique advantages, and the choice of approach should be made in accordance with the specific requirements of the problem being solved.

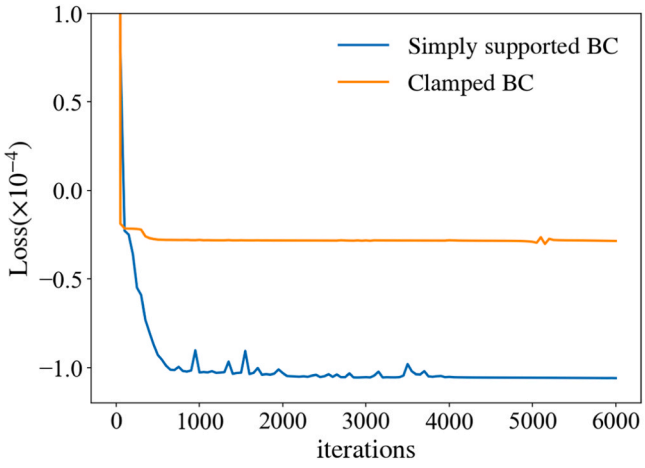



**Fig. 16.** Loss convergence history.

### 6.4. Bending of L-shaped plate

As shown in Fig. 15, an L-shaped plate subjected to a uniform load $q_0 = 1$ N/m$^2$ is considered. The material properties are: Elastic modulus E = 10920000 Pa, Poisson's ratio $\nu = 0.3$. And the thickness $t = 0.01$ m. $a = 0.5$ m. The boundary condition is either simply or clamped support. The following distance function is developed using R-function for the scenario where all edges are simply supported:

$$\phi_1 = (\omega_1 \wedge \omega_2) \wedge (\omega_3 \vee \omega_4),$$

where

$$\left.\begin{array}{l} \omega_1 = x(x-2a) \geq 0 \\ \omega_2 = y(y-2a) \geq 0 \\ \omega_3 = (a-x) \geq 0 \\ \omega_4 = (a-y) \geq 0 \end{array}\right\}$$

For the case where all boundaries are clamped, there is the following distance function:

$$\phi_2 = [(\omega_1 \wedge \omega_2) \wedge (\omega_3 \vee \omega_4)]^2$$

In this example, the original problem domain is divided into 3 subdomains. We set $3 \times 41 \times 41$ training nodes for discretization of the subdomains. Fig. 16 depicts the training losses under these two boundary conditions.

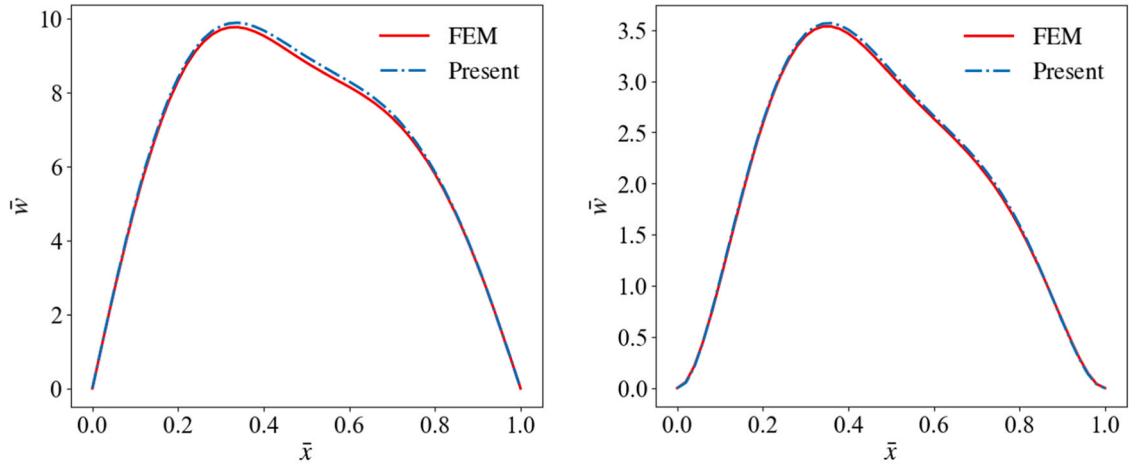



**Fig. 17.** Dimensionless deflections along the axis $\bar{y} = 0.25$.

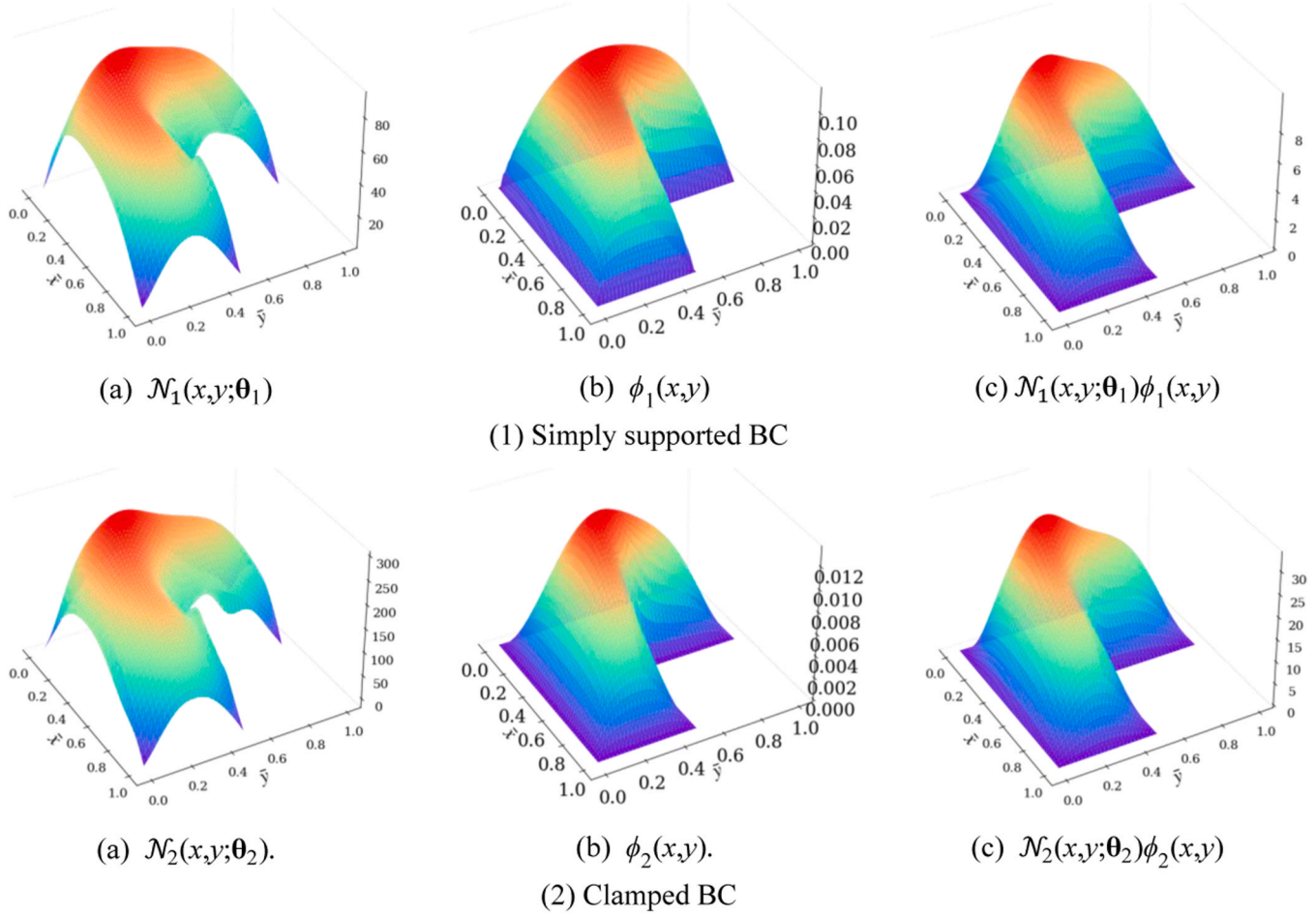



**Fig. 18.** The deflection results in (c) are given by neural network's output in (a) multiplied by the distance function in (b).

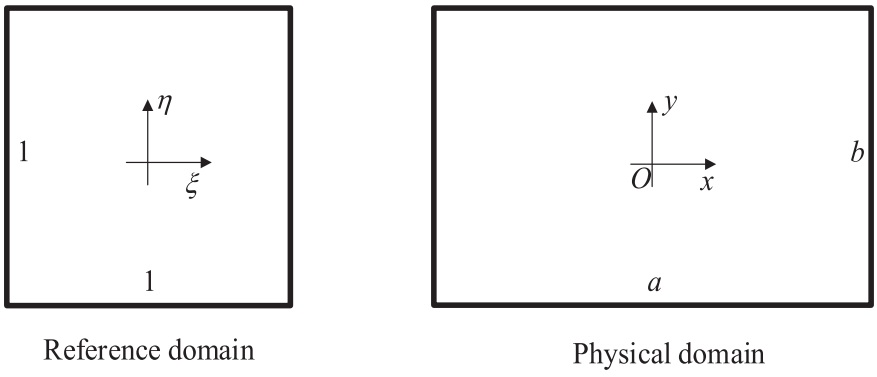



**Fig. 19.** Reference domain and physical domain for rectangular plate.

We use formulas $\overline{w} = 10^3 Dw/(q_0 a^4)$, $\overline{x} = x/(2a)$, $\overline{y} = y/(2a)$ to obtain the dimensionless results. The results along axis $\overline{y} = 0.25$ given by the presented method and FEM are given in Fig. 17. The FEM analysis is run on ABAQUS platform using 3850 STRI3 elements. Both deflection solutions are in good consistency, indicating the feasibility of applying our method to L-shaped plate problems. To better comprehend the method, as shown in Fig. 18, under distance functions $\phi_1(x,y)$ and $\phi_2(x,y)$, what we do is to respectively train two neural network models, $\mathcal{N}_1(x,y;\boldsymbol{\theta}_1)$ and $\mathcal{N}_2(x,y;\boldsymbol{\theta}_2)$, to minimize the loss function in this example.

### 6.5. Free vibration of rectangular plates

This example verifies the effectiveness of the proposed method in free vibration analysis of rectangular Kirchhoff plates (Fig. 19). The plates are simply supported at four edges (SSSS). The material properties: Young's modulus $E = 1.47 \times 10^8$ Pa, Poisson's ratio $\nu = 0.3$, density $\rho = 10^4$ kg/m$^3$. The geometric properties: thickness $t = 0.01$ m, length $a$, width $b = 1$ m. Different $a/b$ ratios are considered. One subdomain is used. To eliminate the boundary conditions, the distance function is selected as:

**Table 6**
Convergence studies of the dimensionless fundamental natural frequencies $\lambda^{'} = \omega b^2/(2\pi)\sqrt{\rho t/D}$.

| Method | $N_{int}$ | $a/b$ | | | | |
|---|---|---|---|---|---|---|
| | | 1 | 1.5 | 2 | 2.5 | 3 |
| Present ($N_l = 4$, $N_n = 20$) | 11 × 11 | 2.9507 | 2.1485 | 1.9270 | 1.7926 | 1.9065 |
| | 21 × 21 | 3.1274 | 2.2527 | 1.9562 | 1.8157 | 1.7409 |
| | 31 × 31 | 3.1353 | 2.2648 | 1.9597 | 1.8205 | 1.7433 |
| | 41 × 41 | 3.1382 | 2.2666 | 1.9618 | 1.8207 | 1.7441 |
| Present ($N_l = 4$, $N_n = 30$) | 11 × 11 | 3.0012 | 2.2261 | 1.8490 | 1.7942 | 1.7087 |
| | 21 × 21 | 3.1198 | 2.2590 | 1.9564 | 1.8158 | 1.7402 |
| | 31 × 31 | 3.1374 | 2.2642 | 1.9597 | 1.8204 | 1.7430 |
| | 41 × 41 | 3.1379 | 2.2666 | 1.9617 | 1.8206 | 1.7441 |
| Present ($N_l = 5$, $N_n = 20$) | 11 × 11 | 2.9502 | 2.2805 | 1.8768 | 1.7956 | 1.8965 |
| | 21 × 21 | 3.1254 | 2.2569 | 1.9552 | 1.8175 | 1.7678 |
| | 31 × 31 | 3.1346 | 2.2644 | 1.9600 | 1.8194 | 1.7429 |
| | 41 × 41 | 3.1382 | 2.2666 | 1.9618 | 1.8206 | 1.7440 |
| Theory [1] | | 3.1416 | 2.2689 | 1.9635 | 1.8221 | 1.7453 |

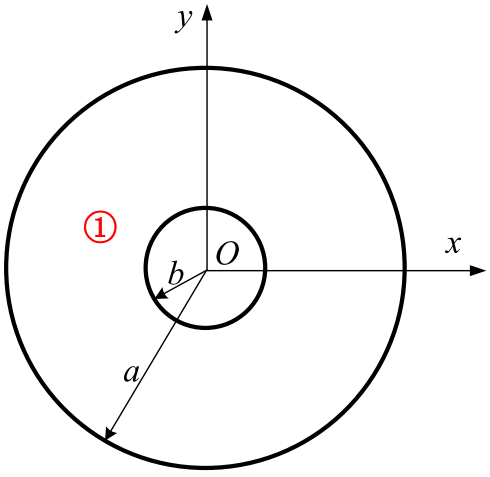



**Fig. 20.** Annular plate.

$$\phi = \left(x^2 - \frac{a^2}{4}\right)\left(y^2 - \frac{b^2}{4}\right).$$

In Table 6, by changing the configuration of integral points, convergence studies are carried out to depict the convergence behavior under different network structures. Different node integral schemes are employed. It can be seen that the adopted networks can achieve good results with sufficient integration nodes.

### 6.6. Free vibration of annular plates

Annular plates (Fig. 20) with free inner edge and clamped outer edge are under consideration in this example. The geometry properties are: outer radius $a$= 1 m, inner radius $b$, thickness $t$ = 0.01 m. The material properties are the same as the previous example. The fundamental natural frequencies of the plate are solved under different $b/a$ ratios.

One subdomain is used. To eliminate the boundary conditions, the distance function is selected as

$$\phi = \left(a^2 - x^2 - y^2\right)^2.$$

Convergence studies of the dimensionless fundamental natural frequencies($\lambda^{'} = \omega a^2/(2\pi)\sqrt{\rho t/D}$.) are carried out along with the results given by ABAQUS. And in ABAQUS analysis, 2500, 2500, 2398, 2046, 2021 and 1719 STRI3 elements are adopted for b/a of 0.1,0.2,0.3,0.4,0.5 and 0.6, respectively. As shown in Fig. 21, a more complicated network is required to properly describe the plate's vibration behavior for the situation of $b/a$= 0.1. Overall, increasing the number of integral nodes can improve the accuracy of the presented method.

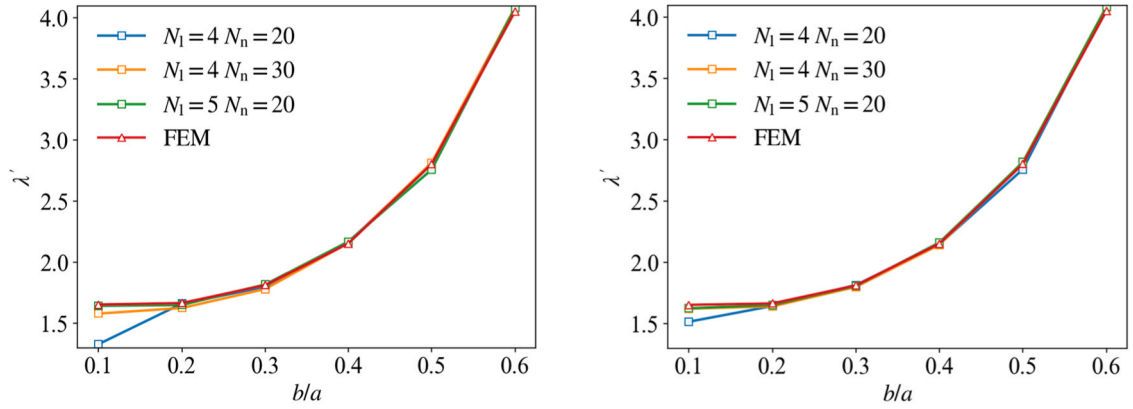
In Table 7, we compare the results given by proposed method under the configuration of $N_l = 5$, $N_n = 20$ with those given by ABAQUS(FEM). The results are in good agreement. And the fundamental mode shapes are given in Fig. 22.

### 6.7. Buckling of skew plates

In the buckling analysis, we calculate the loss according to Eq. (48) during the training process. This example verifies the effectiveness of the proposed method in buckling analysis of skew Kirchhoff plates under unidirectional compression (Fig. 23). The material properties and thickness of the plates are consistent with the plate in Section 6.2. And the edges are all clamped or simply supported. For case of $a$=$b$= 1 m, the buckling factors ($k = \frac{b^2\lambda(\mathbf{x};\mathbf{0})}{\pi^2}$) are calculated under the prediction of

**Table 7**
Comparison of fundamental natural frequencies (Hz) given by the presented method and FEM.

| $b/a$ | Present | FEM | Relative errors |
|---|---|---|---|
| 0.1 | 1.6310 | 1.6555 | -1.50% |
| 0.2 | 1.6561 | 1.6675 | -0.69% |
| 0.3 | 1.8101 | 1.8145 | -0.24% |
| 0.4 | 2.1648 | 2.1564 | 0.39% |
| 0.5 | 2.8188 | 2.8049 | 0.49% |
| 0.6 | 4.0860 | 4.0536 | 0.79% |




(a). 21×21×1 integration nodes (b). 41×41×1 integration nodes

**Fig. 21.** Convergence studies under different integral node configurations.

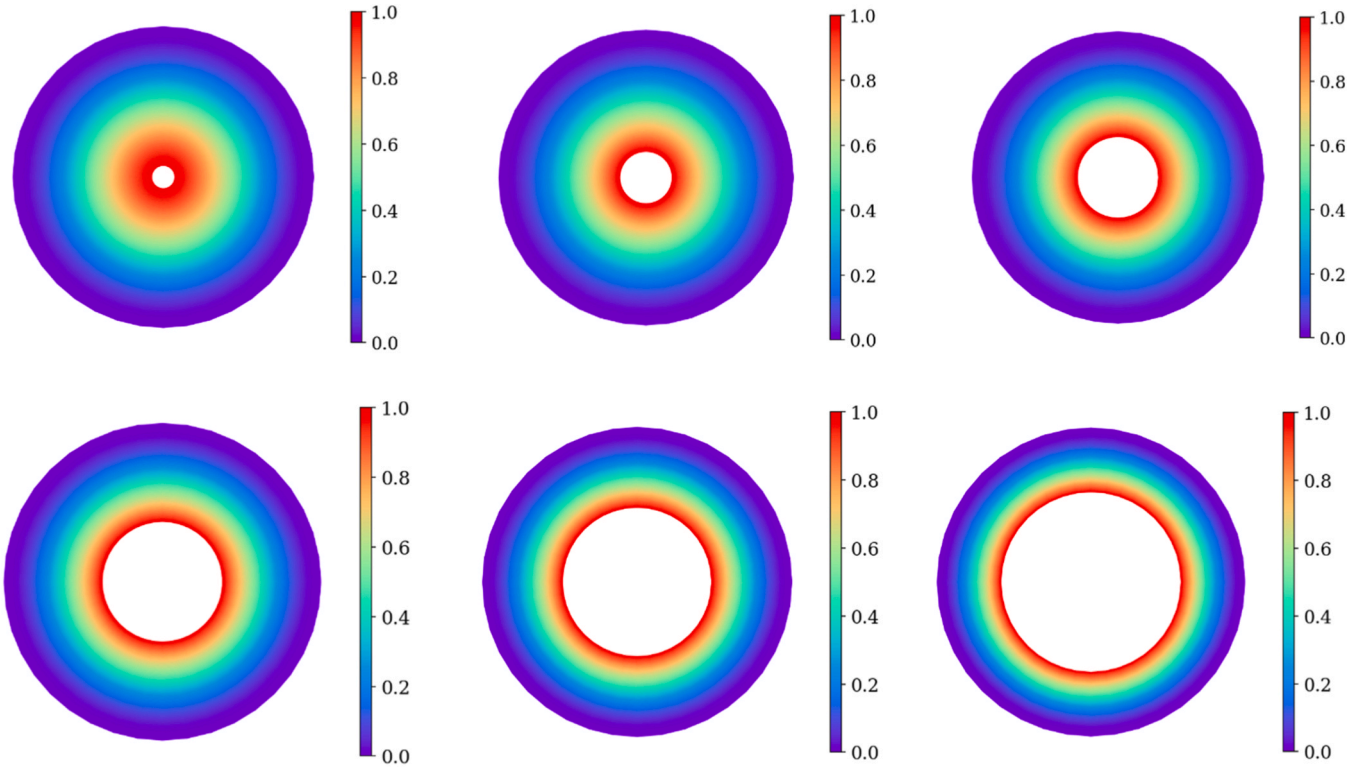



**Fig. 22.** Fundamental mode shapes given by presented method.

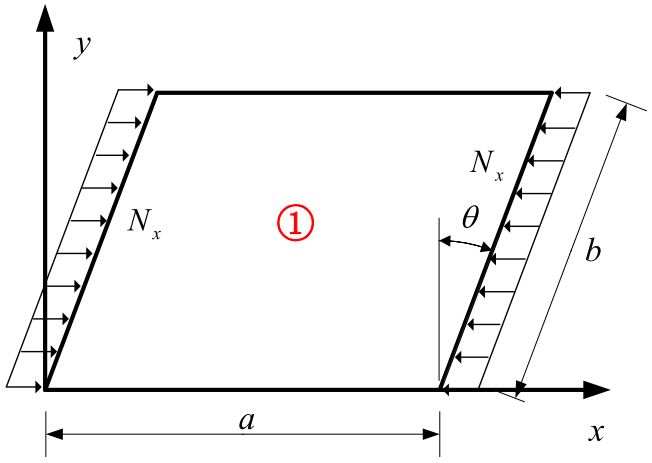



**Fig. 23.** Skew plate.

neural network, and the convergence results are given in Tables 8 and 9. It can be found that the results given by the presented method are consistent with those from the literature in the buckling analysis. In this case, the calculation of coordinate transformation follows a method similar to finite element analysis. Therefore, to a comparable extent, as the skew angle increases, the parallelogram region may experience excessive deformation, leading to an abnormal aspect ratio of the elements. This, in turn, can introduce some degree of numerical instability, resulting in differences among the various solutions presented in Tables 8 and 9, particularly when dealing with larger skew angles. As the skew angle continues to increase, similar to the previous examples, one possible way is to decompose the parallelogram solution domain into multiple subdomains for computation.

**Table 8**
Buckling factors k for simply supported skew plates with α = 1.

| Method | $N_{int}$ | Skew angle $\theta$ | | | |
|---|---|---|---|---|---|
| | | 0° | 15° | 30° | 45° |
| Present | 11 × 11 | 3.9709 | 4.0644 | 5.7696 | 12.3728 |
| | 21 × 21 | 3.9880 | 4.3760 | 6.0210 | 10.0193 |
| | 31 × 31 | 3.9978 | 4.4007 | 5.9828 | 11.0983 |
| | 41 × 41 | 3.9976 | 4.3994 | 6.0619 | 10.8909 |
| Durvasula [45] | | 4.00 | 4.48 | 6.41 | 12.3 |
| Wang [44] | | 4.00 | 4.44 | 6.19 | 10.60 |

**Table 9**
Buckling factors $k$ for clamped skew plates with $\alpha = 1$.

| Method | $N_{int}$ | Skew angle $\theta$ | | | |
|---|---|---|---|---|---|
| | | 0° | 15° | 30° | 45° |
| Present | 11 × 11 | 7.7742 | 8.8668 | 10.3527 | 16.4195 |
| | 21 × 21 | 10.1953 | 10.9609 | 13.9901 | 21.9632 |
| | 31 × 31 | 10.1300 | 10.8936 | 13.8422 | 20.7563 |
| | 41 × 41 | 10.0605 | 10.8585 | 13.6352 | 20.5253 |
| Durvasula [43] | | 10.08 | 10.87 | 13.58 | 20.44 |
| Wang [44] | | 10.08 | 10.89 | 13.75 | 20.69 |

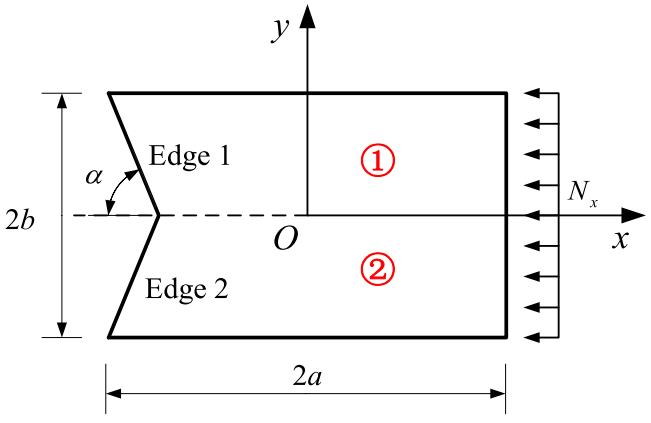



**Fig. 24.** Irregular plates controlled by $\alpha$.

**Table 10**
Results of buckling load factors $\lambda$ under $a$=$b$= 1 m.

| Method | Present | | ABAQUS | |
|---|---|---|---|---|
| | Approach 1 | Approach 2 | STRI3 element | S3 element |
| $\alpha = 30°$ | 8.7376 | 8.9943 | 8.4599 (526) | 8.6474 (526) |
| $\alpha = 45°$ | 1.8360 | 1.8668 | 1.8252 (686) | 1.8981 (686) |
| $\alpha = 60°$ | 1.0017 | 0.9983 | 0.9979 (802) | 1.0146 (802) |
| $\alpha = 80°$ | 0.0675 | 0.6394 | 0.6698 (800) | 0.6744 (800) |

The numbers in parentheses indicate the number of elements in finite element analysis.

### 6.8. Buckling of irregular plates

As shown in Fig. 24, the plate is symmetry about the $x$-axis. The material properties: $E = 1.092 \times 10^7$ Pa, Poisson's ratio $\nu = 0.3$. Edges 1 and 2 are clamped while the remaining edges are free. The thickness $t = 0.01$ m. The plate is subjected to compressive forces in $x$ direction.

**Table 11**
Results of buckling load factors $\lambda$ under $a$= 1 m, $b$= 0.5 m.

| Method | Present | | ABAQUS | |
|---|---|---|---|---|
| | Approach 1 | Approach 2 | STRI3 element | S3 element |
| $\alpha = 30°$ | 1.7237 | 1.6825 | 1.6290 (583) | 1.6734 (583) |
| $\alpha = 45°$ | 0.9727 | 0.9636 | 0.9569 (631) | 0.9795 (631) |
| $\alpha = 60°$ | 0.7460 | 0.7379 | 0.7467 (661) | 0.7582 (661) |
| $\alpha = 80°$ | 0.6101 | 0.6052 | 0.6189 (650) | 0.6224 (650) |

We divide the domain into 2 subdomains. 41 × 41 training nodes are used for each subdomain.

According to R-function theory, the following distance function is obtained

$$\phi = (\omega_1 \vee \omega_2),$$

in which

$$\left.\begin{aligned} \omega_1 &= y + \tan\alpha\left(x + a - \frac{b}{\tan\alpha}\right) \geq 0 \\ \omega_2 &= \tan\alpha\left(x + a - \frac{b}{\tan\alpha}\right) - y \geq 0 \end{aligned}\right\}$$

And two approaches are used to enforce the clamped boundary conditions. Approach 1 employs the method described in Section 5.2, and a portion of the virtual nodes' deflection is calculated according to the difference schemes at the boundary. In approach 2, we modify the distance function to meet the clamped boundary conditions. The modified distance function is $\phi^{'}(x,y) = \phi^2(x,y)$. The buckling load factors $\lambda$, defined in Section 4.4.2, are calculated with these approaches and ABAQUS. The results are listed in Tables 10 and 11, and the STRI3 and S3 shell elements are adopted in finite element analysis. Good agreements between the results given by the proposed method and ABAQUS under different $\alpha$ can be observed. And the buckling mode shapes given by the presented method under approach 1 are plotted in Tables 12 and 13. The presented method exhibits good numerical stability with different geometrical parameters $\alpha$, $a$ and $b$, under different approaches to boundary condition implementation.

**Table 12**
Buckling mode shapes of the irregular plate given by the presented method with different α under a=b= 1 m.

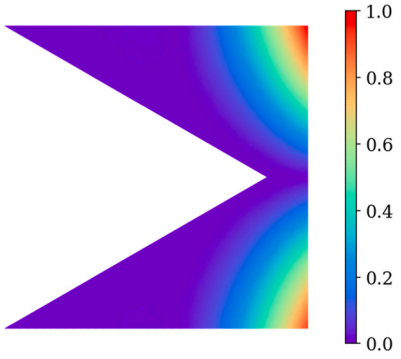
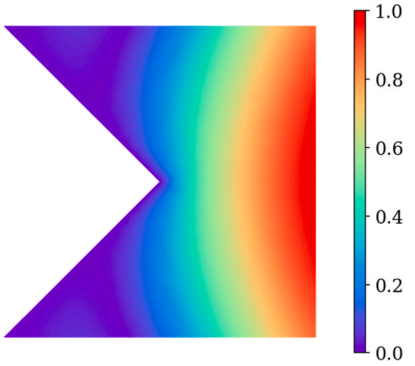
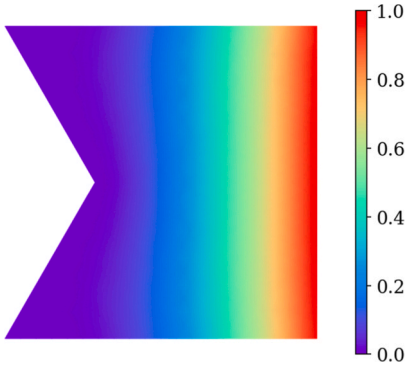
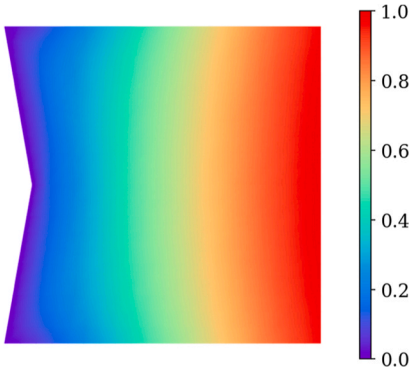
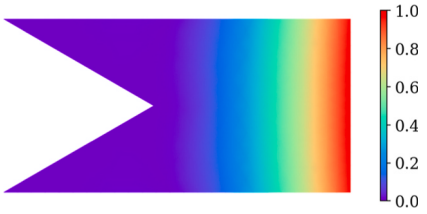
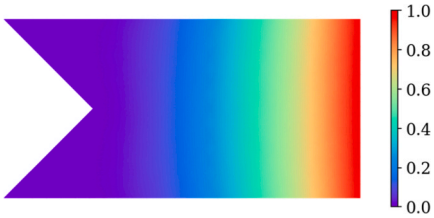
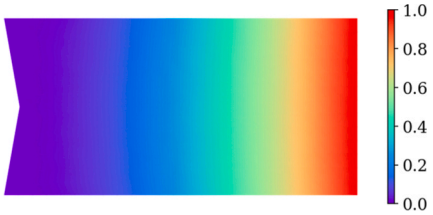
| $\alpha$ | Buckling mode shape | $\alpha$ | Buckling mode shape |
|---|---|---|---|
| 30° |  | 45° |  |
| 60° |  | 80° |  |

**Table 13**
Buckling mode shapes of the irregular plate given by the presented method with different $\alpha$ under $a$= 1 m, $b$= 0.5 m.

| $\alpha$ | Buckling mode shape | $\alpha$ | Buckling mode shape |
|---|---|---|---|
| 30° |  | 45° |  |
| 60° |  | 80° |  |

## 7. Conclusions

An improved plate deep energy method has been developed and applied to analyze the bending, free vibration, and buckling of Kirchhoff plates with irregular shape. In this method, the neural network is trained and works within the physical domain. The primary conclusions are as follows:(1) The presented method enhances the training efficiency. An idea is presented to compute the training losses by utilizing well-designed convolutional kernels and convolution operations on the network's output to approximate the loss function, reducing computational complexity. (2) Our approach can be penalty free. By incorporating the R-function theory, the Dirichlet boundary conditions can be applied to irregular boundaries while also enabling the imposition of derivative boundary conditions through virtual nodes within the reference domain.

The accuracy and efficiency of the proposed method are comprehensively validated through numerical examples, involving a comparison of the results with theoretical solutions, finite element method (FEM) outcomes, and solutions found in the existing literature. It demonstrates the potential of the deep energy approach and the robust fitting capabilities of neural networks in solving partial differential equations in irregular solution domains. Furthermore, our approach can be extended to address high-order derivative boundary conditions and nonlinear problems, promising new avenues for future research in this domain.

## CRediT authorship contribution statement

**Peng Lin-Xin:** Funding acquisition, Project administration, Supervision. **Huang Zhongmin:** Conceptualization, Data curation, Methodology, Writing – original draft, Writing – review & editing.

## Declaration of Competing Interest

The authors declare that there have no conflicts of interest to this work.

## Data Availability

Data will be made available on request.

## Acknowledgements

The work that is described in this paper has been supported by the grants awarded by the National Natural Science Foundation of China (No. 12162004, 11562001), the National Key R&D Program of China (No. 2019YFC1511103) and the Guangxi Science and Technology Project (No. AB22036007).

## References

[1] Ventsel E.S., Krauthammer T., Carrera E. Thin Plates and Shells: Theory: Analysis, and Applications. 2001.
[2] Wu C, Hung Y. Three-dimensional free vibration analysis of functionally graded graphene platelets-reinforced composite toroidal shells. Eng Struct 2022;269:114795.
[3] Civalek Ö, Avcar M. Free vibration and buckling analyses of CNT reinforced laminated non-rectangular plates by discrete singular convolution method. Eng Comput 2022;38:489–521.
[4] Tran V, Nguyen T, Nguyen PTT, Vo TP. Stochastic collocation method for thermal buckling and vibration analysis of functionally graded sandwich microplates. J Therm Stresses 2023;46:909–34.
[5] Civalek Ö. A four-node discrete singular convolution for geometric transformation and its application to numerical solution of vibration problem of arbitrary straight-sided quadrilateral plates. Appl Math Model 2009;33:300–14.
[6] Civalek Ö, Dastjerdi S, Akgöz B. Buckling and free vibrations of CNT-reinforced cross-ply laminated composite plates. Mech Based Des Struct MACHINES 2022;50:1914–31.
[7] Civalek Ö. Vibration of functionally graded carbon nanotube reinforced quadrilateral plates using geometric transformation discrete singular convolution method. Int J Numer Methods Eng 2020;121:990–1019.
[8] Khlifi K, Casimir JB, Akrout A, Haddar M. Dynamic stiffness method: new levy's series for orthotropic plate elements with natural boundary conditions. Eng Struct 2021;245:112936.
[9] Huang ZM, Peng LX. A coordinate transformation based barycentric interpolation collocation method and its application in bending, free vibration and buckling analysis of irregular Kirchhoff plates. Int J Numer Methods Eng 2023;124:5069–101.
[10] Bathe KJ. Finite Element Procedures: Finite Element Procedures. Prentice-Hall; 1996.
[11] Heinrich B. Finite Difference Methods on Irregular Networks. 1987.
[12] Bertoluzza S, Naldi G. A wavelet collocation method for the numerical solution of partial differential equations. Appl Comput Harmon Anal 1996;3:1–9.
[13] Katsikadelis JT. The Boundary Element Method for Plate Analysis. Oxford: Academic Press; 2014.
[14] Bellman R, Casti J. Differential quadrature and long-term integration. J Math Anal Appl 1971;34:235–8.
[15] Lee H, Kang IS. Neural algorithm for solving differential equations. J Comput Phys 1990;91:110–31.
[16] Lagaris I, Likas A, Fotiadis D. Artificial neural networks for solving ordinary and partial differential equations. Neural Netw, IEEE Trans 1998:987–1000.

[17] McFall KS, Mahan JR. Artificial neural network method for solution of boundary value problems with exact satisfaction of arbitrary boundary conditions. IEEE Trans Neural Netw 2009;20:1221–33.
[18] Sirignano J, Spiliopoulos K. DGM: A deep learning algorithm for solving partial differential equations. J Comput Phys 2018;375:1339–64.
[19] Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J Comput Phys 2019;378:686–707.
[20] Yang L, Meng X, Karniadakis GE. B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data. J Comput Phys 2021;425:109913.
[21] Ren P, Rao C, Liu Y, Wang J, Sun H. PhyCRNet: Physics-informed convolutional-recurrent network for solving spatiotemporal PDEs. Comput Methods Appl Mech Eng 2022;389:114399.
[22] Tang S, Feng X, Wu W, Xu H. Physics-informed neural networks combined with polynomial interpolation to solve nonlinear partial differential equations. Computers Math Appl 2023;132:48–62.
[23] Lee JY, Jang J, Hwang HJ. opPINN: Physics-informed neural network with operator learning to approximate solutions to the Fokker-Planck-Landau equation. J Comput Phys 2023;480:112031.
[24] Weinan E, Yu B. The deep ritz method: a deep learning-based numerical algorithm for solving variational problems. Commun Math Stat 2018;6:1–12.
[25] Samaniego E, Anitescu C, Goswami S, Nguyen-Thanh VM, Guo H, Hamdia K, et al. An energy approach to the solution of partial differential equations in computational mechanics via machine learning: concepts, implementation and applications. Comput Methods Appl Mech Eng 2020;362:112790.
[26] Nguyen-Thanh VM, Anitescu C, Alajlan N, Rabczuk T, Zhuang X. Parametric deep energy approach for elasticity accounting for strain gradient effects. Computer Methods Appl Mech Eng 2021;386:114096.
[27] Nguyen-Thanh VM, Zhuang X, Rabczuk T. A deep energy method for finite deformation hyperelasticity. Eur J Mech - A/Solids 2020;80:103874.
[28] Fuhg JN, Bouklas N. The mixed Deep Energy Method for resolving concentration features in finite strain hyperelasticity. J Comput Phys 2022;451:110839.
[29] Abueidda DW, Koric S, Al-Rub RA, Parrott CM, James KA, Sobh NA. A deep learning energy method for hyperelasticity and viscoelasticity. Eur J Mech - A/Solids 2022;95:104639.
[30] He J, Abueidda D, Abu Al-Rub R, Koric S, Jasiuk I. A deep learning energy-based method for classical elastoplasticity. Int J PLASTICITY 2023;162:103531.
[31] He J, Chadha C, Kushwaha S, Koric S, Abueidda D, Jasiuk I. Deep energy method in topology optimization applications. ACTA MECHANICA 2023;234:1365–79.
[32] Guo H, Zhuang X, Alajlan N, Rabczuk T. Physics-informed deep learning for three-dimensional transient heat transfer analysis of functionally graded materials. COMPUTATIONAL Mech 2023.
[33] Bastek J, Kochmann DM. Physics-Informed Neural Networks for shell structures. Eur J Mech - A/Solids 2023;97:104849.
[34] Zhuang X, Guo H, Alajlan N, Zhu H, Rabczuk T. Deep autoencoder based energy method for the bending, vibration, and buckling analysis of Kirchhoff plates with transfer learning. Eur J Mech - A/Solids 2021;87:104225.
[35] Roy AM, Guha S. A data-driven physics-constrained deep learning computational framework for solving von Mises plasticity. Eng Appl Artif Intell 2023;122:106049.
[36] Niu S, Zhang E, Bazilevs Y, Srivastava V. Modeling finite-strain plasticity using physics-informed neural network and assessment of the network performance. J Mech Phys Solids 2023;172:105177.
[37] Shukla K, Jagtap AD, Karniadakis GE. Parallel physics-informed neural networks via domain decomposition. J COMPUTATIONAL Phys 2021;447:110683.
[38] Gao H, Sun L, Wang J. PhyGeoNet: physics-informed geometry-adaptive convolutional neural networks for solving parameterized steady-state PDEs on irregular domain. J Comput Phys 2021;428:110079.
[39] Chiu P, Wong JC, Ooi C, Dao MH, Ong Y. CAN-PINN: A fast physics-informed neural network based on coupled-automatic–numerical differentiation method. Computer Methods Appl Mech Eng 2022;395:114909.
[40] Sukumar N, Srivastava A. Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. Computer Methods Appl Mech Eng 2022;389:114333.
[41] V.L. Rvachev, Theory of R-Functions and Some Applications, Naukova Dumka, Kiev. In Russian, 1982.
[42] V. Shapiro, Theory of R-functions and applications: A primer, Tech. Rep. CPA88–3, Cornell Programmable Automation, Sibley School of Mechanical Engineering, Ithaca, NY 14853, USA, 1991.
[43] Durvasula S. Buckling of clamped skew plates. AIAA J 1970;8(1):178–81.
[44] Wang CM, Liew KM, Alwis W. Buckling of skew plates and corner condition for simply supported edges. J Eng Mech 1992;118(4):651–62.
[45] Durvasula S. Buckling of simply supported skew plates. J Eng Mech 1971;97:967–79.
[46] Kingma D, Ba J. Adam: a method for stochastic optimization. Comput Sci 2014.
[47] Glorot X, Bengio Y. Understanding the difficulty of training deep feedforward neural networks. J Mach Learn Res 2010;9:249–56.
[48] Wang L, Zhao J. Computation graph. In: Wang L, Zhao J, editors. Architecture of Advanced Numerical Analysis Systems: Designing a Scientific Computing System using OCaml. Berkeley, CA: Apress; 2023. p. 149–89. https://doi.org/10.1007/978-1-4842-8853-5_6.